专业课问题汇总

1. 前言

计算机专业课考察几乎在绝大部分学校的面试中都会遇到，考察各位同学的专业基础和数理基础，主要考察内容包括计算机核心课程（408），数学知识，机器学习等等。专业课复习的过程中，可以主要复习概念，算法等，大部分面试中均不要求实际求解某一问题。本文主要描述面试中 408 常考题型，具体原理理解可参考王道考研教学视频。

数据结构：王道计算机考研_数据结构_哔哩哔哩_bilibili

计算机网络：王道计算机考研_计算机网络_哔哩哔哩_bilibili

操作系统：王道计算机考研_操作系统_哔哩哔哩_bilibili

计算机组成原理：王道计算机考研_计算机组成原理_哔哩哔哩_bilibili

2. 计算机核心课程（408）

计算机核心课程 408 包括计算机网络，计算机组成原理，数据结构，操作系统。

2.1. 数据结构

1. O(n)的大 O 是什么意思？什么是时间复杂度？ ★★★

O(n)这个大 O 表示的是最坏情况下的时间复杂度即渐进时间复杂度，即当 n 趋于无穷大时，算法所需要的计算工作量，对程序规模的一个描述。

2. 线性表的顺序存储结构和链式存储结构的优点★★★

从存储空间，访问情况，以及环境三方面来看。首先，**存储空间**：线性结构物理和逻辑结构一致，事先分配连续的一大片空间，因而存储密度较大，与此相反，链表可动态分配空间，要存储下一个节点地址因为存储密度较低；从**访问情况**来看，线性结构是连续的，支持随机访问，但如果有增加和删除操作时，需要移动大量元素，效率较低，而链式结构需要遍历整个表，因而不支持随机访问，

但是增删操作很方便；从**环境**来看，线性表易于实现，且任何高级语言都支持数组类型，而链表基于指针，更复杂，不一定都支持。

3. 解释一下顺序存储与链式存储★★★

顺序存储结构吧逻辑上相邻的元素存放到物理位置上相邻的存储单元中，数据元素之间的逻辑关系由存储单元的邻接位置关系来体现。

链接存储方法不要求逻辑上相邻的元素在物理位置上也相邻，元素之间的逻辑关系由附加的指针指示。

4. 头指针和头结点的区别？★★

以单链表为例，头指针就是指向链表第一个结点的指针，链表的第一个结点的地址可以通过链表的头指针找到，对单链表中任一元素的访问必须首先根据头指针找到第一个结点。

而头结点是一个附加结点，头结点的 data 域可以不存储任何信息，也可以存放特殊标志或表长，只要表存在就必须至少有一个头结点。**头结点的存在可以优化单链表的增删操作**，使得对于空表或者在非空表第一个结点之前插入不用作为特殊情况专门处理，而是使用通用方法。

5. 栈和队列的区别和内存结构 ★★★

栈的定义是只允许在表的末端进行插入和删除的线性表，允许插入和删除的一端叫做栈顶，不允许插入和删除的一端叫做栈底。

栈的存储结构通常分为基于数组的存储表示和顺序存储结构和基于链表的链式存储结构，

顺序栈的内存结构：存放栈中元素的数组、栈顶指针、最大容纳元素个数

链式栈的内存结构：栈顶指针

队列的定义是：只允许在表的一端插入，在另一端删除的线性表。允许插入元素的一端称为队尾，允许删除的一端称为队首。

队列的存储结构通常分为基于数组的存储表示和基于链表的存储表示。

顺序存储结构利用一个一维数组作为存储结构，并设置头指针和尾指针两个指针来指示队首和队尾的位置。

链式队列由队首指针和队尾指针构成。

6. 有一个循环队列 Q，里面的编号是 0 到 n-1，头尾指针分别是 f，p，现在求 Q 中元素的个数？★★

(p-f+n)%n

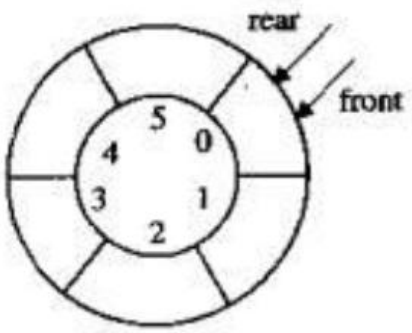
7. 如何区分循环队列是队空还是队满？★★★

front 表示队头指针（指向队列内首元素）

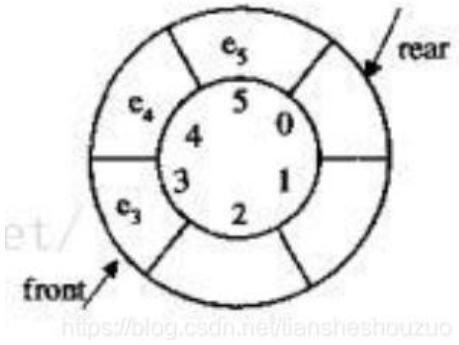
rear 表示队尾指针（指向队列内尾元素的下一个位置）

m 表示队列的容量（包括那个留空的位置）

队空：front=rear



队满：front=(rear+1)%m



8. 堆、大顶堆、小顶堆实现及应用 ★★

- 堆：非线性结构，可以看成数组或者完全二叉树，**通俗来说是利用完全二叉树的结构来维护一维数组，但并非完全二叉树。**
- 大顶堆：节点大于等于其左右子树的节点
- 小顶堆：节点小于等于其左右子树的节点

应用：Topk 问题、求中位数、堆排序（不稳定）、优先级队列的应用场景（如：定时任务轮询和合并有序小文件）

9. 哈希表的概念、构造方法、哈希有几种类型？哈希冲突的解决办法？★★★★

- 哈希表：根据设定的哈希表和所选中的处理冲突的方法，**将一组关键字映像到一个有限的地址区间上。**
- 哈希表构造方法：**直接地址法、平方取中法、除留余数法**
- 哈希的冲突解决：开放地址法、再哈希法、链地址法、建立公共溢出区

https://www。nowcoder。com/tutorial/93/e7dc954667634630b6c1e93ac87dd271

- **哈希表的概念：**哈希表是一种使用哈希函数组织数据，以支持快速插入和搜索的数据结构。
- **哈希有几种类型：**分为哈希集合和哈希映射，哈希集合是集合数据结构的实现之一，用于存储非重复值。哈希映射是映射 数据结构的实现之一，用于存储(key, value)键值对。
- **构造方法：**对于构造哈希来说，主要包括**直接地址法、平方取中法、除留余数法**等。
- **处理哈希冲突**，最常用的方法有**开放寻址法、再哈希法、链地址法**

10. 判断链表是否有环（非常重要！）★★★★★★★

快慢指针：从头开始设置两个指针，快指针每次走 2 步，慢指针每次走 1 步，如果快指针先碰到尾，则无环，否则两个指针之后一定会重合，则有环。（leetcode 原题）

11. 平衡二叉树、二叉排序树、完全二叉树、二叉搜索树的区别及如何构造★★

- 完全二叉树的每一个节点都与高度为 k 的满二叉树的 1-n 编号一一对应。
- 满二叉树： 树中每个分支结点（非叶结点）都有两棵非空子树
- 二叉搜索树和二叉排序树是一个东西。
- 二叉排序树每一个节点都是用于排序的关键码，左子树所有节点关键码都小于根节点的关键码，右子树的所有关键码都大于根节点的关键码，其中，左右子树都是二叉树。对一棵二叉排序树进行中序遍历就可以得到关键码从小

到大的排列。

- **二叉排序树的构造方法主要执行插入操作**，进行插入之前必须先检查是否该节点的关键码已经在树中存在，也就是要先查找，假如查找成功，不执行任何操作，假如搜索不成功，就在搜索停止的地方添加新元素。
- 平衡二叉树是一种二叉搜索树，它的左右子树高度差的绝对值不超过 1。**构造一棵平衡二叉树仍然是通过插入节点的方式，每插入一个结点，都应该检查平衡因子，并通过旋转操作使之平衡化。**

12. 如何由遍历序列构造一棵二叉树？已知先序序列和后序序列能否重现二叉树？（笔试经常考）★★★

- 若是中序和先序或者中序和后序，则可。
- 首先序中我们可以锁定当前第一个点为根，中序中找到这个点，去把子树分割成左子树和右子树两个部分，再依次递归下去即可 https://leetcode-cncom/problems/construct-binary-tree-from-preorder-and-inorder-traversal/
- 先序和后序不能重现一棵唯一的二叉树。但有可能找出满足此的二叉树。

在知道后序和中序遍历的原理之后，我们也可以反推出一颗树。

思路：

- 1）后序遍历的最后一位即是二叉树的根节点。
- 2）然后再中序遍历找到根结点，根节点左边的所有序列便是左子树的中序遍历结果，右边的所有序列便是右子树中序遍历结果。
- 3）在后序遍历中找到对应的左子树，右子树。
- 4）在左右子树的先序和中序结果中继续重复1， 2， 3步骤，便可还原二叉树。

13. B 树是什么？在数据库中有什么应用？（B 数和 B+树的区别）★★★

1. 二叉搜索树：二叉树，每个结点只存储一个关键字且值大于左子树，小于右子树。

2. B（B-）树：多路搜索树，每个结点存储M/2到M个关键字，非叶子结点存储指向关键字范围的子结点； 所有关键字在整颗树中出现，且只出现一次，非叶子结点可以命中；

3. B+树：在B-树基础上，为叶子结点增加链表指针，所有关键字都在叶子结点中出现，非叶子结点作为叶子结点的索引；B+树总是到叶子结点才命中；

在数据库查询中，以树存储数据，**B 树的一个结点可以装多个值，所有的值在树中只出现一次，**读取时，是把整个结点读到内存，然后在内存中，对结点的值进行处理，在内存中处理速度肯定比磁盘快。所以只要树的高度低，IO 少，就能够提升查询效率，这是 B 树的好处之一。

B 树的缺点是：**不利于范围查找(区间查找)**，如果要找 0~100 的索引值，那么 B 树需要多次从根结点开始逐个查找。

B+树和 B 树最大的不同是：B+树内部有两种结点，一种是索引结点，一种是叶子结点。B+树的索引结点并不会保存记录，只用于索引，所有的数据都保存在 B+ 树的叶子结点中。而 B 树则是所有结点都会保存数据。

B+树的叶子结点都会被连成一条链表。叶子本身按索引值的大小从小到大进行排序。即这条链表是从小到大的

在数据库查询中，以树存储数据。树有多少层，就意味着要读多少次磁盘 IO。所以树的高度越矮，就意味着查询数据时，需要读 IO 的次数就越少。（众所周知，读 IO 是一件费时的操作）当数据量大的时候，用 AVL 树（平衡二叉树）存的话，就算 AVL 是平衡树，但是也扛不住数据量大，**数据量大，AVL 树的树高肯定很**高，那么读取数据的 IO 次数也会多。**B 树的一个结点可以装多个值**，读取时，是把整个结点读到内存，然后在内存中，对结点的值进行处理，在内存中处理速度肯定比磁盘快。所以只要树的高度低，IO 少，就能够提升查询效率，这是 B 树的好处之一。

B 树的缺点是：**不利于范围查找(区间查找)**，如果要找 0~100 的索引值，那么 B 树需要多次从根结点开始逐个查找。

B+树和 B 树最大的不同是：B+树内部有两种结点，一种是索引结点，一种是

叶子结点。B+树的索引结点并不会保存记录，只用于索引，所有的数据都保存在B+树的叶子结点中。而 B 树则是所有结点都会保存数据。

B+树的叶子结点都会被连成一条链表。叶子本身按索引值的大小从小到大进行排序。即这条链表是 从小到大的。因此可以直接通过遍历链表实现范围查找。

(48 条消息) B 树、B-树、B+树、B*树介绍_Zzzzzzzz_hu 的博客-CSDN 博客_b 树

参考 CSDN：深入理解 Mysql 索引底层原理 - 知乎 (zhihu.com)

14. 红黑树原理是什么？建立过程？★★★

红黑树：**本质上来说就是一棵二叉查找树**，红黑树的 5 条性质：

1）每个结点要么是红的，要么是黑的。

2）根结点是黑的。

3）每个叶结点（叶结点即指树尾端 NIL 指针或 NULL 结点）是黑的。

4）如果一个结点是红的，那么它的两个儿子都是黑的。

5）对于任一结点而言，其到叶结点树尾端 NIL 指针的每一条路径都包含相同数目的黑结点。

正是红黑树的这 5 条性质，使得一棵 n 个结点是红黑树始终保持了 logn 的高度，从而也就解释了上面我们所说的“红黑树的查找、插入、删除的时间复杂度最坏为 O(log n)”这一结论的原因。

15. 二分搜索和单纯的线性搜索的区别/时间复杂度★★★

二分要求是有序表，必须是顺序存储，ASL 小，O(logn)；二线性搜索不要求顺序，可以是顺序也可以链式存储，O(n)

16. 插入排序、希尔排序、选择排序、冒泡排序、归并排序、快速排序、堆排序、基数排序等排序算法的基本思想是什么？时间复杂度？是否稳定？给一个例子，问冒泡和快速排序在最坏的情况下比较几次？（排序必考）★★★★★★

- **插入排序**：目标是构建有序序列，默认第一个有序，然后将无序序列中的第一个元素与有序序列比较放到合适的位置，直到整个序列有序；**是稳定的**。
- **希尔排序**：是简单插入排序的改进版。它与插入排序的不同之处在于，它会

优先比较距离较远的元素。希尔排序又叫缩小增量排序。按照增量 d 将序列分成若干子序列，对每个子序列直接进行插入排序，d 依次减小到 1 最终实现排序；**不稳定**。

- **选择排序**：每次找到序列中的最小值依次放入序列中，具体来说第一次遍历找到最值交换到队头构成有序队列，在从无序队列中找最值交换到无序队列中的对头，依此类推得到整个有序序列；**不稳定的**。
- **冒泡排序**：从左往右依次比较，如果顺序错误交换，有序队列在队尾构建；**稳定的**。
- **归并排序**：建立在归并操作上，利用分治的思想，先使每个子序列有序，再将子序列合并成有序表，以二路归并为例，将长度为 n 的输入序列分成两个长度为 n/2 的子序列，对每个子序列在采用归并排序，将两个有序的子序列**合并成有序序列；稳定的**。
- **快速排序**：以一个关键字为基准，在一次排序后，将序列划分成两部分，左边的都比其小，右边比其大，再分别对两个子序列做快排实现整个序列有序；**不稳定的**。
- **堆排序**：以堆为数据结构，在构建大顶堆或小顶堆的过程中实现升序或者降序排序。具体以升序序列来说，先将无序序列构建成大顶堆，再将堆顶与堆尾元素交换，再对前 1~n-1 个元素重复上述过程直至整个有序；**不稳定。**
- **基数排序**：借助多关键字排序的思想来实现单关键字排序，主要是最高位优先和最低位优先两种，为节约空间以链式结构为主，具体来说，可以使用分配和收集的策略，先使最低位关键字有序，再次关键字直到主关键字有序；**稳定的，O(nk)**。
- **计数排序**：找到最大值和最小值，将序列映射到最大值-最小值+1 那么长的区间中，采用分配收集策略，扫描一遍原数组，将每个元素-最小值作为新数组的下标并将其+1，收集时扫描一遍计数器数组，按值填充新数组；**稳定的，O(n+k)**。
- **桶排序**：在计数排序的基础上，找到最值划分区间，子序列排好序后合并；**稳定的，O(n+k)**。

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(nlog_2n)$	$O(n^2)$	$O(nlog_2n)$	$O(nlog_2n)$	不稳定
归并排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均情况	最好情况	最坏情况	辅助存储	
插入排序	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	Shell排序	$O(n^{1.3})$	$O(n)$	$O(n^2)$	$O(1)$	不稳定
选择排序	直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(n^2)$	$O(nlog_2n)$	不稳定
归并排序		$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(n)$	稳定
基数排序		$O(d(r+n))$	$O(d(n+rd))$	$O(d(r+n))$	$O(rd+n)$	稳定

注：基数排序的复杂度中，r代表关键字的基数，d代表长度，n代表关键字的个数。

CSDN @青年有志

基本思想参考：https://www。cnblogs。com/onepixel/articles/7674659。html

冒泡排序最坏情况：如 4，3，2，1 升序排序，需要比较 3+2+1=6=4*3/2 次

快速排序最坏情况：已经排好顺序的（包括升序、降序、所有元素都一样）最坏的情况：每次找的 pivot 将数组分成两部分，其中有一部分是空，这样递归树就变成了一棵倾斜的树。树的深度为 n-1，这样时间复杂度就变成了 O(N^2).

17. **最小生成树和最短路径用什么算法来实现？（迪杰斯特拉、弗洛伊德、普利姆、克鲁斯卡尔）算法的基本思想是什么？算法的时间复杂度？如何进行优化？★★★★★★★**

最小生成树：普利姆 Prim、克鲁斯卡尔 Kruskal

- Prime：核心思想是将顶点集划分为生长点集合和非生长点集合，每次从非生长点集合中挑选一个代价最小的可以连接两个集合的边，该条边必然是合法边；具体来讲，随机初始化一个点为生长点集合，在非生长点集合中找到所有待选边中值最小的加入到生长点集合中并更新两个集合及对应的边，直到非生长点集合为空；O(n^2)，**适合边稠密的图。**
- Kruskal：考虑生成树边权最小，应该使每条边权值都尽可能地小，具体来说，先构造一个只含 n 个顶点子图，选择权值最小且不会产生回路的边（判断是否有环，可以使用拓扑排序和深度搜索），直至加满 n-1 条边为止；O(eloge)，**适合边稀疏的图。**
- **最短路：迪杰斯特拉、弗洛伊德、SPFA，单源负权边时是** Bellman Ford 算法，迪杰斯特拉，克鲁斯卡尔，普利姆是贪心。
- Dijkstra：**贪心思想，只能求正权值（出现负权值时，由于之前已经确定好了最短路径的点不会再次更新，按照深度更新，不会回头），单源点最短路径**；指定源点，计算源点到各个点距离，选取最小距离为源点到当前点的距离，新加入点之后，将新点作为桥梁，计算源点通过新点到其他点的距离和源点之前到各个点的距离，积累最小的边并加入新点，直到所有顶点遍历完；O(n^2)，再找最小遍对应的顶点时可采用最小堆进行维护，此时时间复杂度为 O(nlogn)。
- **弗洛伊德：动态规划，只允许环路的权值为负数**，其他路径的权值必须为非负数，否则算法执行过程会出错，通过枚举中间点，i-j 的距离可以被优化成经过中间点 i，k--k，j---i，j，O(n^3)
- SPFA：**可以处理负权图（按照广度搜索更新，松弛操作会入队列，有机会被再次更新，因而可以计算负权），无法处理带负环的图**（环路的边权总和小于 0，会一直绕圈子更新；但可以在第 n 次判断是否存在负权环），指定源点入队列，取出队首顶点，更新点到各个点的距离，如果距离有变动且新点未在队列中，则将其加入到队列中，重复上述过程，和 Bellman-ford 的区

别在于使用队列来维护已经进行过松弛操作的节点避免冗余计算（新的松弛操作必然发生在最短路径节点前已经松弛成功的节点，将其入队列记录下来）;不稳定的算法,最好时优于 Dijkstra,最坏时 O(VE)直逼 Bellman-ford，一般不存在负权时选用 Dijkstra;

18. 邻接表和邻接矩阵（如何存储大数据）★

- 邻接矩阵：用于记录顶点间的邻接关系，**开销较大，适合稠密图**;
- 邻接表：将所有顶点按序存储到结构体数组中，结构体有两个域，第一个用于存值，第二个为指针域，初始为空，指向该点的所有边对应的顶点；适用于稀疏图;
- 稀疏图与稠密图的区别：定点不是决定性因素，主要看边，有的顶点多但是度少，现实生活中大部分为稀疏图。

19. 介绍一下深度优先搜索和广度优先搜索是如何实现的？★★★

- 深度优先搜索：将当前节点的直接子节点作为候选节点进行操作，并递归的搜索其直接子节点直到子节点为空，再回溯到父节点，搜索未操作过的直接子节点，重复上述操作直到所有节点得到遍历；选择最新成为候补的顶点，沿着一条路径搜索到底。**运用栈或者递归实现**
- 广度优先搜索：将当前节点的直接子节点直接作为候选子节点，并按照先后顺序依次遍历其直接子节点，因而采用队列维护；选择最早成为候补的节点，沿着边搜索；**运用队列实现**

20. 介绍一下字符串匹配算法：朴素的匹配算法和 KMP 算法。（如何实现要会用语言描述）★★★

- **朴素匹配 BF 算法**：暴力匹配，最容易想到，从源字符串的第一个字符开始搜索，出现不匹配则从原位置+开始重新进行搜索；简单易于理解，时间复杂度为 O(n*m)。

- **过渡**：最大的时间损失来自于源串的回退，为了尽量减少子串与主串的匹配次数，如何找到一种方法在失配后能够使得主串指针不回退，而只移动子串指针；直观来看，主串和子串在当前位置但在其之前所有位置都是匹配的，如果找到已匹配的子串中前缀和后缀匹配度最大的元素，那么将前缀的尾元素对其适配元素前一位置，直接比较主串当前位置和子串最大前缀尾元素的下一个位置即可。
- **KMP 算法**：核心先预处理计算一个失配表，记录各个位置失配时应该将子串指针调整到哪个位置；整个过程，匹配主串和子串，若匹配两者指针+1，失配主串不动，子串查失配表回退到相应位置。求失配表的过程，是一个匹配问题，实际上就是原问题，只不过主串和子串都是自己，假设 i，j 为主串和子串指针，如果 i 和 j 所在位置匹配，则匹配长度为 j+1（j 可以看成已经匹配的长度），反之 j=next[j]。

```
void get_next(SString T,int next[ ]){
    i=1;next[1]=0;j=0;
    while (i<=T[0]) {
      if (j= =0||T[i]==T[j]) {++i; ++j; next[i]=j;}
      else  j=next[j];
    }
  }// get_next
```

时间复杂度O(m)

模式匹配：KMP算法

```
int Index_KMP(SString S,SString T,int pos){
    i=pos;j=1;
    while (i<=S[0]&&j<=T[0]) {
      if (j= =0||S[i]==T[j]) {++i;++j;}
      else  j=next[j];
    }
    if (j>T[0]) return i -T[0];
    else return 0;
}// Index_KMP
```

时间复杂度O(n)

21. 二叉树的基本形态有几种？

5 种，分别是：空树、只有根节点、左子树为空、右子树为空、左右子树均不为空。**二叉树与树的关系？**二叉树并非树的特殊情况，这是因为二叉树分左右子树，树不分左右。**二叉树与度为二叉树的关系？**二叉树子树分左右，即使只有一棵子树；**度为 2 的有序树是一棵二叉树？**错误，因为有序树在只有一个孩子时，不分左右。

一般地：有n个结点的不相似二叉树有：

$$\frac{1}{n+1}C_{2n}^{n}$$

2.2. 计算机网络

1. OSI 和 TCP/IP 模型各个层之间的协议和功能★★★★★★

OSI模型	TCP/IP模型	功能	协议
应用层	应用层	为应用程序提供服务	建立在TCP上：FTP、SMTP、TELNET、HTTP 建立在UDP上：DNS、SNMP、NFS、TFTP
表示层		数据格式化、数据加密	无
会话层		建立或解除与别的接点的联系	无
传输层	传输层	提供端对端的接口	TCP、UDP
网络层	网络层	IP选址与路由选择	IP、ICMP、IGMP
数据链路层	链路层	传输有地址的帧以及错误检测功能	ARP、RARP
物理层		以二进制数据形式在物理介质上传输数据	ISO2210、IEEE802

- 物理层：透明传输比特流
- 数据链路层：封装成帧，透明传输，差错检测（CRC）
- 网络层：负责为分组交换网上的不同主机提供通信服务，**IP 选址和路由选择**
- 传输层：负责向两个主机中进程之间的通信提供服务，分用和复用，拥塞控制，流量控制，可靠传输（差错控制）
- 应用层：为应用程序提供服务

“透明”是一个很重要的术语。它表示，某一个实际存在的事物看起来却好像不存在一样。“在数据链路层透明传送数据”表示无论什么样的比特组合的数据都能够通过这个数据链路层。因此，对所传送的数据来说，这些数据就“看不见”数据链路层。或者说，数据链路层对这些数据来说是透明的。

2. 计算机网络为什么要分层？优点？ ★★★

优点：

1. 便于维护和拓展
2. 各层之间相互独立，灵活性好
3. 结构上可以区分开，当某层不需要时，甚至可以去掉
4. 能够促进标准化工作

总的来说，封装对下层的变化。一般来说一个复杂的多功能的程序，为了**便**

于维护与拓展，程序的功能之间是解耦合的，计算机网络也是这样。通过分层进行封装解耦，将复杂的大问题分解成若干个容易解决的小问题，且**各层之间相互独立**，不需要知道下一层如何实现，通过层间接口提供或者访问服务即可；**灵活性好**，即使某一层内部发生变化也不影响其他层调用；如此看来，**结构上可分割开的，当某层不需要时甚至可以将其取消**，每层可以采用最合适的技术，正因如此每层的服务已经明确说明，可以**促进标准化工作**。

1.各层之间是独立的。某一层并不需要知道它的下一层是如何实现的，而仅仅需要知道该层通过层间的接口所提供的服务。由于每一层只实现一种相对独立的功能，因而可将一个难以处理的复杂问题分解为若干个较容易处理得更小一些的问题。这样，整个问题的复杂程度就下降了。

2.灵活性好。当任何一层发生变化时（例如由于技术的变化），只要层间接口关系保持不变，则在这层以上或以下各层均不受影响。此外，对某一层提供的服务还可进行修改。

3.当某层提供的服务不再需要时，甚至可以将这层取消。

4.结构上可分割开。各层都可以采用最合适的技术来实现。

5.易于实现和维护。这种结构使得实现和调试一个庞大而又复杂的系统变得易于处理，因为整个的系统已被分解为若干个相对独立的子系统。

6.能促进标准化工作。因为每一层的功能及其所提供的服务都已有了精确的说明。

3. 简述一下层次路由的原理（叙述一下与自治系统相关的内部网关协议和外部网关协议）单工、半双工、全双工通信？★★

路由包括：直连路由、静态路由、动态路由，下面均为动态路由：

为什么划分 AS（自治系统）？互联网规模非常大，如果路由表要存储所有路由显然不可行，且许多单位并不想让别人知道其内部的布局细节，因而划分区域自治系统。现在一般都是 AS 内部采用内部网关协议，如：RIP、OSPF；AS 间通信采用外部网关协议，如：BGP。

- RIP：基于距离向量（可以理解为最短跳数），UDP 传送，在固定的时间间隔内和相邻路由器交换路由表；算法简单，易于实现，适用于小型网；坏消息传得慢，有条数限制，更新较慢。

- OSPF：最短路径优先（使用 Dijkstra，求单源点最短路径），IP 数据报传送，当链路状态变化时，采用洪范法和其他路由器交互链路状态信息，最终可以得到链路拓扑图，但只会存储下一跳地址；灵活可指派代价，可以实现负载均衡，洪范法能够快速收敛，实现较为复杂；
- BGP：基于路径向量，**要求可达而非最佳路径**（因为出于安全政策等因素），各个 AS 选择 BGP 代言人，代言人之间通过建立 TCP 链接实现 BGP 会话以交换路由信息，一轮过后就能找出较好的路由；首次交换整个路由表，之后有变化才交换；实现较简单。

协议	RIP	OSPF	BGP
类型	内部	内部	外部
路由算法	距离-向量	链路状态	路径-向量
传递协议	UDP	IP	TCP
路径选择	跳数最少	代价最低	较好，非最佳
交换结点	和本结点相邻的路由器	网络中所有的路由器	和本结点相邻的路由器
交换内容	当前本路由器知道的全部信息，即自己的路由表	与本路由器相邻的所有路由器的链路状态	首次：整个路由表；非首次：有变化的部分

4. 协议三要素？（语法、语义、时序）★★

- 协议：计算机网络中为了进行数据交换建立的规则集合
- 语法：表示要怎么做，**规定数据**的一些结构格式
- 语义：要做什么，**要发出何种信息，做出何种动作，何种响应**
- 时序：做的顺序，事件顺序的说明

5. 香农公式？奈氏准则？信道容量含义？带宽增加，信道容量怎么变？香农公式的前提条件？

- 奈氏准则：在理想条件下（带宽受限无噪声），指出码元传输速率是有上限的，超过此上限会出现严重的码间串扰；

- 香农公式：指出了在有噪声的条件下，**信道的极限传输速率和信噪比和带宽有关系**，如果两者都达到了极限，**想要提高速率只能通过让每个码元携带更多的比特信息**（换一种编码方式）。
- 信道容量：反映了信道所能传输的最大信息量，带宽增加若其他因素不变，信道容量变大。

6. 简述一下 CSMA/CD 协议★★★★

协议的实质是“载波监听”和“碰撞检测”。先听后发，边发边听，冲突停发，随机延迟后重发（截断二进制指数退避算法）

CSMA 协议(载波侦听多路访问)（Carrier Sense Multiple Access）

- **非持续式：**经侦听，如果介质空闲，开始发送；如果介质忙，则等待一个随机分布的时间，然后重复

  优点：等待一个随机时间可以减少再次碰撞冲突的可能性

  缺点：如果在这个随机时间内介质上没有数据传送，则会发生浪费
- **1-持续式：**经侦听，如介质空闲，开始发送；如介质忙，持续侦听，一旦空闲立即发送；如果发生冲突，等待一个随机分布的时间再重复步骤。

  优点：持续式的延迟时间要少于非持续式

  缺点：如果两个以上的站等待发送，一旦介质空闲就一定会发生冲突
- **p-持续式：**经侦听，如介质空闲，那么以 p 的概率发送，以(1－p)的概率延迟一个时间单元发送；如介质忙，持续侦听，一旦空闲重复步骤；如果发送已推迟一个时间单元，再重复步骤

CSMA/CD 协议（Collision Detection：碰撞检测）

载波侦听多路访问 / 碰撞检测(Carrier Sense Multiple Access with Collision Detection)

协议是 CSMA 协议的改进方案。“载波帧听”就是发送前先侦听，即每个站在发送数据之前先要检测一下总线上是否有其他站点正在发送数据，若有则暂时不发送数据，等待信道变为空闲时再发送。“碰撞检测”就是边发送边侦听，即适配器边发送数据边检测信道上信号电压的变化情况，以便判断自己在发送数据时其他站点是否也在发送数据。工作流程可简单概括为“**先听后发，边听边发（区**

别于 CSMA 协议），冲突停发，随机重发”。

1）适配器从其父结点获得一个网络层数据报，准备一个以太网帧，并把该帧放到适配器缓冲区中。

2）如果适配器侦听到信道空闲，那么它开始传输该帧。如果适配器侦听到信道忙，那么它将等待直至侦听到没有信号能量，然后开始传输该帧。

3）在传输过程中，适配器检测来自其他适配器的信号能量。如果这个适配器传输了整个帧，而没有检测到来自其他适配器的信号能量，那么这个适配器完成该帧的传输（碰撞一定会发生在前 64 个字节，如果没碰撞之后也不可能碰撞，所以要求 IP 数据报长度不小于 46 字节，因为头部控制信息有 18 字节，如果少于则填充字节）。否则，适配器就须停止传输它的帧，取而代之传输一个 48 比特的拥塞信号。

4）在中止（即传输拥塞信号）后，适配器采用截断二进制指数退避算法（确定 r 个争用期即端到端的往返时延，r 由[1，2，……，2^k-1]中的一个随机数，k=min(重传次数，10)）等待一段随机时间后返回到步骤 2)。

7. TCP 和 UDP 的异同点★★★★★★

◆ **TCP 和 UDP 的相同点：**TCP 和 UDP 都是传输层协议，都能保护网络层的传输，双方的通信都需要开放端口。

◆ **TCP 和 UDP 的不同点：**

1）TCP 的传输是可靠传输；UDP 的传输是不可靠传输。

2）TCP 是**基于连接**的协议，在正式收发数据前，必须和对方建立可靠的连接。UDP 是和 TCP 相对应的协议，它是**面向非连接**的协议，它不与对方建立连接，而是直接把数据包发送出去

3）TCP 是一种可靠的通信服务，负载相对而言比较大，**TCP 采用套接字（socket）或者端口（port）来建立通信；UDP 是一种不可靠的网络服务，负载比较小。**

4）TCP 和 UDP 结构不同，TCP 包括序号、确认信号、数据偏移、控制标志（通常说的 URG、ACK、PSH、RST、SYN、FIN）、窗口、校验和、紧急指针、选项等信息；UDP 包含长度和校验和信息。

5）**TCP 提供超时重发**，丢弃重复数据，检验数据，流量控制，拥塞控制等功能，保证数据能从一端传到另一端；UDP **不提供可靠性**，它只是把应用程序传给 IP 层的数据报发送出去，但是并不能保证它们能到达目的地。

6）TCP 在发送数据包前在通信双方有一个**三次握手机制**，确保双方准备好，在传输数据包期间，TCP 会根据链路中数据流量的大小来调节传送的速率，传输时如果发现有丢包，会有**严格的重传机制**，故而**传输速度很慢**；UDP 在传输数据报前**不用**在客户和服务器之间建立一个**连接**，且没有超时重发等机制，故而传输**速度很快**。

7）**TCP 支持全双工和并发的 TCP 连接**，提供确认、重传与拥塞控制，对**数据完整性**要求较高；UDP 适用于哪些系统**对性能的要求高**于数据完整性的要求，需要“简短快捷”的数据交换、需要**多播和广播**的应用环境。

8）TCP 面向字节流，UDP 面向数据报

原文链接：https://blog.csdn.net/quiet_girl/article/details/5059977

8. *TCP 的三次握手四次挥手过程？为什么会采用三次握手，若采用二次握手可以吗？★★★★★

四次挥手:https://blog.csdn.net/guoweimelon/article/details/50879302

三次握手:https://blog.csdn.net/guoweimelon/article/details/50878730

三次握手对每次发送的数据量进行跟踪与协商

1. 客户端向服务器发起建立连接请求
2. 服务端收到后，像客户端回复一个建立连接请求的响应
3. 客户端收到后，继续向服务端发送一个响应
4. 三次握手完成，正式建立连接。

四次挥手：

1. 断开发起方向另一方发送断开连接请求
2. 另一方收到后，回复一个响应
3. 再由另一方主动发送一个断开连接请求
4. 断开发起方收到后，回复一个响应

四次挥手完成，断开 SOCKET 连接。

如果二次握手：

先假如出现了一种异常情况，即 A 发出的第一个连接请求报文段因为在某些网络节点上滞留了。由于超时重传，于是 A 又向 B 发起请求并成功建立了连接，在传输完数据之后，AB 同之间释放了连接。

而在 **A 和 B 已经释放连接之后，那个在网络上滞留的报文段又达到了 B**。这时候，B 接收到报文以为是 A 发起的新的一次建立连接的请求，于是就向 A 发出确认建立连接报文段。而 A 此时并没有发起建立连接的请求，于是不予理睬。但是 B 以为新的连接已经建立，一直等待 A 发送数据，于是 B 的许多资源就浪费了。

9. 介绍下 TCP 和 UDP 协议的特点、头部结构★★★★★

TCP 和 UDP 的头部均有：源端口和目的端口，和校验和信息（二进制反码求和，包括首部+伪首部+数据），伪首部即 IP 首部

TCP 头部结构：序号、确认号、数据偏移、控制标志（通常说的 URG、ACK、PSH、RST、SYN、FIN）、窗口、校验和、紧急指针、选项等信息；

UDP 头部结构：长度

10. *简述下 TCP 建立连接的过程，TCP 如何保证可靠传输？★★★★★

TCP 协议保证数据传输可靠性的方式主要有：

◆ 校验和：TCP 与 UDP 都通过二进制反码求和对首部+伪首部+数据进行检验

◆ 序列号+确认号：序列号对每个传输字节编号。确认好对收到的数据进行应答，告诉接收方哪些数据已经收到，并告知期望的数据开始序号；

◆ 超时重传：ARQ 重传机制，连续 ARQ 和滑动窗口机制等，保证数据一定能被收到；

◆ 连接管理：TCP 连接的建立是实现可靠传输的前提，三次握手和四次挥手确保连接的成功建立和释放；

◆ 流量控制：发送方和接收方速率不匹配的问题，是端到端的控制，通过滑动窗口的大小来控制；

◆ 拥塞控制：是个全局性的过程，设计到所有的主机、路由器，是为了解决避免网络资源被耗尽的问题，避免网络过载。

https://blog.csdn.net/liuchenxia8/article/details/80428157?ops_request_misc=%257B%2522request%255Fid%2522%253A%252216402456351678027415550 3%2522%252C%2522scm%2522%253A%252220140713.130102334.%2522%257D&request_id=164024563516780274155503&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduend~default-1-80428157.pc_search_insert_es_download&utm_term=TCP%E5%A6%82%E4%BD%95%E4%BF%9D%E8%AF%81%E5%8F%AF%E9%9D%A0%E4%BC%A0%E8%BE%93&spm=1018.2226.3001.4187

11. 在 TCP 拥塞控制中，什么是慢开始、拥塞避免、快重传和快恢复算法？★★★★

假设：1. 数据是单方向传递，另一个窗口只发送确认；2. 接收方的缓存足够大，因此发送方的大小由网络的拥塞程度来决定。

一：慢开始算法和拥塞避免算法

当对资源的需求大于了可用资源就会发生拥塞

发送方会维持一个拥塞窗口，刚开始的**拥塞窗口和发送窗口相等**，然后我们每过一个传输轮次，就让拥塞窗口加倍，接着发送分组也是原来的两倍，以此类推，**当窗口值等于 16（慢开始门限）**，然后我们开始采用“加法增大”的策略，转变为每轮次加 1 的方式。直到网络拥塞。我们开始采用“拥塞避免”算法：让新的慢开始门限值**变为发生拥塞时的值的一半**，将拥塞窗口置为 1，然后让它再次重复慢开始算法，这时一瞬间会将网络中的数据量大量降低。

二：快重传和快恢复算法

快重传可以提高网络的吞吐量而**快恢复算法相当于拥塞避免算法的后半恢复部分的优化。**

假设以下情况:如果在发送方设置的超时定时器到时间还没有收到确认，那么有一种可能是网络发生堵塞，这种情况下，**TCP 会将拥塞窗口置为一，新的门限值变为发生阻塞时的一半并且开始执行慢开始算法**。当我们使用快重传的时候，要求接收方接收到一个失序的报文段后就立即发出重复确认，(目的是让对方早知道有报文段没有到达)。

假设发送方发送了 M1 – M4 四个分组，接收方收到了 M1 和 M2，以及 M4，这些分组。现在接收方不能确认 M4，因为 M3 没有收到，此时接收方可以什么都不干，也可以发送对 M2 的确认，但是快重传算法要求这样做：

接收方应该及时发送对 M2 的重复确认，这样可以让发送方知道 M3 并没有被传过来，发送方还会试着发送 M5，M6，接收方收到之后，我们会继续发送对 M2 的确认，这样一共发了好几个对 M2 的确认，按照规定，**只要发送方收到三个重复确认，就立即重传对方未收到的报文段 M3。这样可以避免阻塞**，并且提高我们网络的吞吐量。

三：快恢复算法与快重传算法配合使用：

当发送方收到三个连续确认时，就执行“乘法减小”算法，把“慢开始门限”减半，注意接下来不会执行慢开始算法。

快恢复算法：由于此时没有发送网络阻塞(要是发生阻塞的话就不会连续收到 4 个确认)，因此此时不执行慢开始算法，并不会将拥塞窗口的值置为 1，而是将它**置为慢开始门限的一半**。然后再实行拥塞避免算法每轮次之后+1。

(53 条消息) TCP 拥塞控制：慢开始、拥塞避免、快重传、快恢复_半盏流年的博客-CSDN 博客_慢开始快重传

12. TCP 的快速重传机制★★★★

快重传可以提高网络的吞吐量而**快恢复算法相当于拥塞避免算法的后半恢复部分的优化。**

假设以下情况:如果在发送方设置的超时定时器到时间还没有收到确认，那么有一种可能是网络发生堵塞，这种情况下，**TCP 会将拥塞窗口置为一，新的门限值变为发生阻塞时的一半并且开始执行慢开始算法**。当我们使用快

重传的时候，要求接收方接收到一个失序的报文段后就立即发出重复确认，(目的是让对方早知道有报文段没有到达)。

假设发送方发送了 M1 - M4 四个分组，接收方收到了 M1 和 M2，以及 M4，这些分组。现在接收方不能确认 M4，因为 M3 没有收到，此时接收方可以什么都不干，也可以发送对 M2 的确认，但是快重传算法要求这样做：

接收方应该及时发送对 M2 的重复确认，这样可以让发送方知道 M3 并没有被传过来，发送方还会试着发送 M5，M6，接收方收到之后，我们会继续发送对 M2 的确认，这样一共发了好几个对 M2 的确认，按照规定，**只要发送方收到三个重复确认，就立即重传对方未收到的报文段 M3。这样可以避免阻塞**，并且提高我们网络的吞吐量。

(53 条消息) TCP 滑动窗口/快速重传机制_Hxingwei 的博客-CSDN 博客

13. 流量控制和拥塞是什么关系？★★★★

流量控制解决的是发送方和接收方速率不匹配的问题；拥塞控制解决的是避免网络资源被耗尽的问题。流量控制是通过滑动窗口来实现的；拥塞控制是通过拥塞窗口来实现的。发送方的发送窗口不能超过接收方的接收窗口的数值。

拥塞控制是作用于网络的，它是防止过多的数据注入到网络中，避免出现网络负载过大的情况，涉及到网络中所有的路由器、主机；常用的方法就是：**慢开始、拥塞避免、快重传、快恢复。**

流量控制是作用于接收者的，它是控制发送者的发送速度从而使接收者来得及接收，防止分组丢失的。

https://www.huaweicloud.com/articles/ba31660f82bbdd3a3e8615ec68d85bc3。html

14. 两个服务器之间网络已经联通，却收不到彼此的 UDP 报文原因★★★★

可能的原因很多，比如设置了 acl（访问控制列表），**禁用了某些端口，网络拥塞，丢包**等。

主要丢包原因：

1、**接收端处理时间过长导致丢包：**调用 recv 方法接收端收到数据后，处理数据花了一些时间，处理完后再次调用 recv 方法，在这二次调用间隔里，发过来的包可能丢失。对于这种情况可以修改接收端，将包接收后存入一个缓冲区，然后迅速返回继续 recv。

2、**发送的包巨大丢包：**虽然 send 方法会帮你做大包切割成小包发送的事情，但包太大也不行。例如超过 50K 的一个 udp 包，不切割直接通过 send 方法发送也会导致这个包丢失。这种情况需要切割成小包再逐个 send。

3、**发送的包较大，超过接受者缓存导致丢包：**包超过 mtu size 数倍，几个大的 udp 包可能会超过接收者的缓冲，导致丢包。这种情况可以设置 socket 接收缓冲。以前遇到过这种问题，我把接收缓冲设置成 64K 就解决了。

```
int nRecvBuf=32*1024;//设置为 32K
setsockopt(s， SOL_SOCKET， SO_RCVBUF， (const char*)&nRecvBuf，sizeof(int));
```

4、**发送的包频率太快：**虽然每个包的大小都小于 mtu size 但是频率太快，例如 40 多个 mut size 的包连续发送中间不 sleep，也有可能导致丢包。这种情况也有时可以通过设置 socket 接收缓冲解决，但有时解决不了。所以在发送频率过快的时候还是考虑 sleep 一下吧。

5、**局域网内不丢包，公网上丢包**。这个问题我也是通过切割小包并 sleep 发送解决的。如果流量太大，这个办法也不灵了。总之 udp 丢包总是会有的，如果出现了用我的方法解决不了，还有这个几个方法：要么减小流量，要么换 tcp 协议传输，要么做丢包重传的工作。

15. 地址解析协议和 RARP 协议★★

ARP：根据 IP 地址查找对应的 Mac 地址，每个主机都会有一个 ARP 缓存区，如果有直接发送；如果没有，广播发送 ARP 数据包，里面带有源主机的 IP 地址和 mac 地址，目的主机的 ip 地址，其余主机如果发现 ip 匹配则会将信息其写入自己的 ARP 表中，单播响应 mac 地址，反之如果不匹配直接丢弃。

16.网卡是什么？功能？★★★

1.进行串并转换，计算机内部数据并行，外部串行，由此可见也是局域网和计算机的接口，也可以实现数模转换

2.执行协议（如：IEEE802。3 的以太网帧协议以及解封、CSMA/CD 协议）

3.加载驱动程序

4.缓存数据

5.数据的编码解码，比如：Manchester 编码

网卡是工作在链路层的网络组件，是局域网中连接计算机和传输介质的接口，不仅能实现与局域网传输介质之间的物理连接和电信号匹配，还涉及帧的发送与接收、帧的封装与拆封、介质访问控制、数据的编码与解码以及数据缓存的功能等。

网卡的作用：

网络是通过模拟信号将信息转化为电流传播的，网卡在这里面就充当了一个解码器的作用，将电信号重新转换成文字图像等就是网卡的责任。网卡的其他功能还有监控上传及下载流量，控制网速稳定的作用，它就相当于电脑的港口，所有信息上传到网络之前都要先到网卡这里走一遭。

每台电脑都有网卡，没有网卡无法上网。

17.说下网络中的主机通信流程★★

主要分两种情况，假设主机 A/B 通信：

1.若 A 和 B 在同一网段，通过二层交换机连接，那么 A 想向 B 发消息，消息经过层层封装到链路层时，会查询 ARP 缓存，如果没有对方的 Mac 地址，会发送 ARP 广播求得 mac 地址，得到地址之后封装成 Mac 帧，发送到交换机，**交换机通过源地址学习法会生成地址和端口的对应表（转发[已知某端口]、过滤[AB 都在本端口]、扩散[未知端口]、广播[群发消息]）**，如此走对应端口出去后，B 通过层层解封装收到 A 发送的消息；

2.若 A 和 B 不在同一网段通过三次交换机或路由器相连，则除了上述过程以外还要经过三次交换机或者路由器进行路由转发实现跨网段通信。具体来说，路

由器内部有一张路由表，可以通过直连路由、静态路由、动态路由生成，将目的 IP 地址和路由表项的子网掩码相与得到的值如果和表项的 IP 地址相同，则从指明的 IP 地址转发出去，否则检查是否有特定路由，有就转发，否则检查是否有默认路由，有则转发，如果都没有就丢弃，最终也会到达主机 B。

18. 一个主机将两个端口接到网络上是否会提升吞吐量？为什么？★★★

如果两个端口均单独使用，那就要看每个端口的吞吐量是否平衡。如果某个端口吞吐量很小，那就没什么太大区别。

19. 简述下 DNS 域名解析的过程。★★★★★

首先查询本地 DNS 服务器，没有则将请求发送到根域名 DNS 服务器，然后再更具根域名 DNS 服务器递归的查找下一级域名信息的 DNS 服务器，最后在目标域名的 DNS 服务器上找到相应的 IP 地址信息并返回。

1、当客户机提出查询请求时，首先在本地计算机的缓存中查找。如果在本地无法获得查询信息，则将查询请求首先发给**本地 DNS 服务器**。

2、当本地 DNS 服务器接到查询后，首先在该服务器管理的区域的记录中查找，如果找到该记录，则利用此记录进行解析；如果没有区域信息可以满足查询要求，服务器在本地的缓存中查找。

3、如果本地服务器不能在本地找到客户机查询的信息，将客户机请求发送到**根域名 DNS 服务器**。

4、根域名服务器负责解析客户机请求的根域部分，它将包含**下一级域名信息的 DNS 服务器(一般为顶级域名服务器)**地址返回给客户机的 DNS 服务器地址。

5、客户机的 DNS 服务器利用根域名服务器解析的地址访问**下一级 DNS 服务器（权限域名服务器）**，得到再下一级域我的 DNS 服务器地址。

6、按照上述递归方法逐级接近查询目标，最后在有目标域名的 DNS 服务器上找到相应 IP 地址信息。

7、客户机的本地 DNS 服务器将递归查询结果返回客户机。

8、客户机利用从本地 DNS 服务器查询得到的 IP 访问目标主机，就完成了一

个解析过程。

1. 浏览器搜索自己的DNS缓存（维护一张域名与IP地址的对应表）

2. 若没有，则搜索操作系统中的DNS缓存（维护一张域名与IP地址的对应表）

3. 若没有，则搜索操作系统的hosts文件（Windows环境下，维护一张域名与IP地址的对应表，位置一般在 C:\Windows\System32\drivers\etc\hosts）

4. 若没有，则操作系统将域名发送至 **本地域名服务器- -**（递归查询方式），**本地域名服务器 查询自己的DNS缓存**，查找成功则返回结果，否则，（以下是迭代查询方式）

   4.1 **本地域名服务器** 向根域名服务器（其虽然没有每个域名的具体信息，但存储了负责每个域，如com、net、org等的解析的顶级域名服务器的地址）发起请求，此处，根域名服务器返回com域的顶级域名服务器的地址

   4.2 **本地域名服务器** 向com域的顶级域名服务器发起请求，返回baidu.com权限域名服务器（权限域名服务器，用来保存该区中的所有主机域名到IP地址的映射）地址

   4.3 **本地域名服务器** 向baidu.com权限域名服务器发起请求，得到www.baidu.com的IP地址

5. **本地域名服务器** 将得到的IP地址返回给操作系统，同时自己也将IP地址缓存起来

6. 操作系统将 IP 地址返回给浏览器，同时自己也将IP地址缓存起来

7. 至此，浏览器已经得到了域名对应的IP地址

20. 点击网页一次 HTTP 请求过程？（在浏览器里输入一个网址，会发生什么）★★★

参考链接：https://blog。csdn。net/qq_21993785/article/details/81188253

1. DNS 解析
2. TCP 三次握手建立连接
3. 发起 HTTP 请求
4. **服务器响应 HTTP 请求**（Nginx 进行负载均衡）
5. **浏览器解析**渲染，拿到 HTML 文件后，解析代码构建 DOM 树，遇到静态资源就多线程请求下载，keep-alive（HTTP1.1 才有）特性使得可以建立一次 HTTP 连接而请求下载多个资源；把请求到的资源和代码进行渲染，边解析边渲染
6. **服务器关闭 TCP 连接**，keep-alive 超时会造成 http 连接失活，但是 TCP 连接一般不会主动断掉，而是由客户端发起中断请求。

21. 机器的 ip 地址和 mac 地址，他们有什么区别，有什么用途？★★

1. IP 地址在网络层是动态获取的，基于区域划分，**随着位置变化而动态变化**，mac 地址在链路层是出厂时就已经写死在网卡里的，不随位置的改变而改变；

2. IP 地址的出现，是因为网络的异构性（不同的计算机生产商使得硬件地址体系不同，不同的网络运营商，不同的需求，运行不同的协议等等）是客观存在的，**为了让这些异构的网络能够通过统一的方式进行通信，出现了 IP 协议、IP 地址。**

3. MAC 地址可以唯一标识一台机器，IP 地址因为是动态接入的，在没有分配到 IP 地址时就没有办法区分不同设备

22. HTTP 状态码及其含义 、HTTP 和 HTTPS 的区别★★★★

状态码含义：https://blog.csdn.net/whl826661099/article/details/98606745

200——服务器成功返回网页；404——请求的网页不存在；503——服务器超时；

1XX：临时响应；2XX：成功；3XX：重定向；4XX：请求错误；5XX：服务器错误。

区别：https://www.cnblogs.com/wqhwe/p/5407468.html

HTTP 协议传输的数据都是未加密的，也就是明文的，因此使用 HTTP 协议传输隐私信息非常不安全，为了保证这些隐私数据能加密传输，于是网景公司设计了 SSL（Secure Sockets Layer）协议用于对 HTTP 协议传输的数据进行加密，从而就诞生了 HTTPS。简单来说，HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，要比 http 协议安全。

HTTPS和HTTP的区别主要如下：

1、https协议需要到ca申请证书，一般免费证书较少，因而需要一定费用。

2、http是超文本传输协议，信息是明文传输，https则是具有安全性的ssl加密传输协议。

3、http和https使用的是完全不同的连接方式，用的端口也不一样，前者是80，后者是443。

4、http的连接很简单，是无状态的；HTTPS协议是由SSL+HTTP协议构建的可进行加密传输、身份认证的网络协议，比http协议安全。

23. DHCP 的作用？★★

参考链接：https://baike.baidu.com/item/DHCP/218195?fr=aladdin

用于集中管理、分配 IP 地址，使环境中的主机能够动态的获得 IP 地址、网关地址、DNS 服务器地址。

24. 能不能直接在应用层把数据交给网络层★★★

在应用层数据的呈现形式是声音、图形、文字等具体的信息，而网络通信信道所能传输的是电信号（一般是数字信号），以上信息无法直接通过信道传输，所以，必须将数据转化成二进制传输。

不完整回答（从传输层目的层面）：

传输层为运行在不同主机上的**进程之间**提供了逻辑通信，而网络层则提供了**主机之间**的逻辑通信。

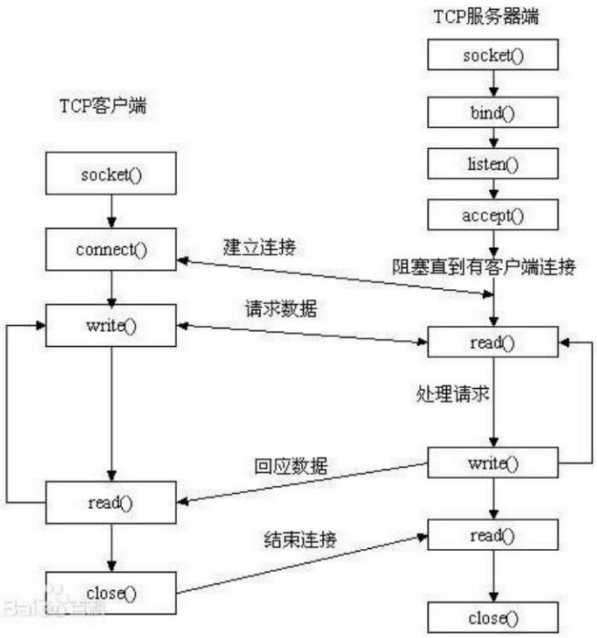
传输层向上提供可靠的和不可靠的逻辑通信信道

TCP 和 UDP 都用端口号来识别应用层实体，以便准确地把信息交给上层对应的协议(进程)。

25. 了解 Socket 吗？什么是 socket？★★★★

网络套接字是 IP **地址与端口的**组合。**Socket 相当于从网络向进程传递数据和 从进程向网络传递数据的门户，它只是提供了一个针对 TCP 或者 UDP 编程的接口。**

socket 其实就是**操作系统提供给程序员操作「网络协议栈」的接口**，说人话就是，你能通过 socket 的接口，来控制协议找工作，从而实现网络通信（提供了应用进程利用网络协议交换数据的机制），达到跨主机通信。



26. 简述一下 Cookie 和 Session 的区别★★★

1、数据存放位置不同：

cookie 数据存放在客户的**浏览器**上，session 数据放在**服务器**上。

2、安全程度不同：

cookie 不是很安全，别人可以分析存放在本地的 COOKIE 并进行 COOKIE 欺骗，考虑到安全应当使用 session。

3、性能使用程度不同：

session 会在一定时间内保存在服务器上。当访问增多，会比较占用你服务器的性能，考虑到减轻**服务器性能**方面，应当使用 cookie。

4、数据存储大小不同：

单个 cookie 保存的数据不能超过 4K，很多浏览器都限制一个站点最多保存 20 个 cookie，而 session 则存储与服务端，浏览器对其没有限制。

5、会话机制不同

session 会话机制：session 会话机制是一种**服务器端机制，**它使用类似于哈希表（可能还有哈希表）的结构来保存信息。

cookies 会话机制：cookie 是服务器存储在本地计算机上的小块文本，**并随每个请求发送到同一服务器**。 Web 服务器使用 HTTP 标头将 cookie 发送到客户端。在客户端终端，浏览器解析 cookie 并将其保存为本地文件，该文件自动将来自同一服务器的任何请求绑定到这些 cookie。

27. [计算机网络之面试常考](https://www.cnblogs.com/zyf-zhaoyafei/p/4716297.html)

https://www.cnblogs.com/zyf-zhaoyafei/p/4716297.html

2.3. 操作系统

1. 操作系统的特点？功能？★

操作系统特点：

操作系统的四个基本特征是**并发，共享，异步，虚拟**。

并发：指计算机系统中“同时”的运行着多个程序，这些程序宏观上是同时运行的，在微观上是交替运行的。

共享：即资源共享（互斥共享、同时共享），是指系统中的资源可供内存中多个并发执行的进程中同时使用。

异步：由于资源等因素的限制，程序运行并不是一气呵成的，而是走走停停的，它以不可预知的速度向前推进。

虚拟：将物理上的实体变为若干个逻辑上的对应物，前者是实际存在的，后者是用户感受到的；又分为时分复用（处理器分时共享）和空分复用（虚拟存储器）。

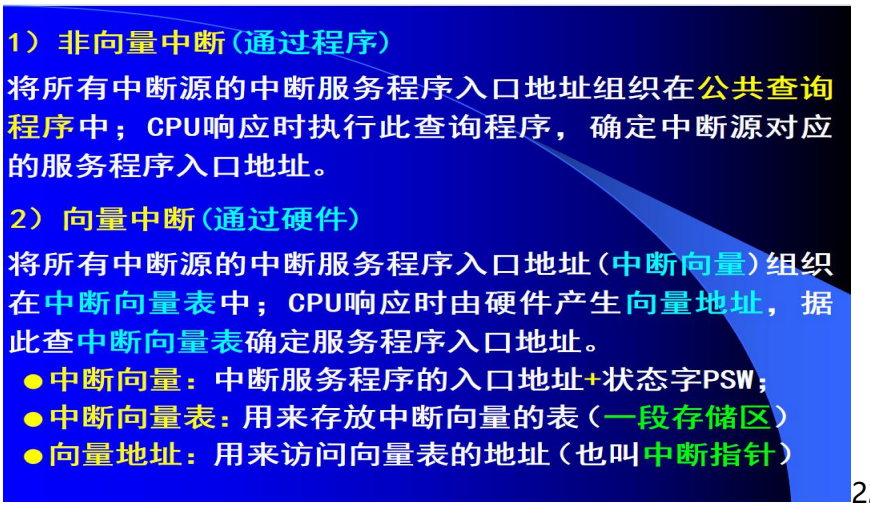
而每个操作系统又有其独特的特征，如我们常用的 linux 系统有开放性；多用户多任务；设备的独立性；强大的网络功能和网络可靠性等特点；操作系统的功能：指计算机系统中“同时”的运行着多个程序，这些程序宏观上是同时运行的，在微观上是交替运行的；管理计算机系统的全部软、硬件资源 ， 合理组织计算机的工作流程 ， 以达到充分发挥计算机资源的效率 ， 为用户提供友好界面

2. 中断、系统调用、库函数调用的区别★★★

中断和系统调用的区别

中断分两种，**硬中断和软中断，中断处理过程一般包括：请求中断——响应中断——保存断点和保护现场——中断源识别(软中断 or 硬中断)——中断服务——恢复现场——中断返回，**执行中断例行程序时，是开中断的，**只有保存断点、保存现场、恢复现场时需要关中断**；硬中断是实实在在的硬件发出的中断，cpu 检测到发生中断后，保护现场，**查找中断向量地址，执行中断服务程序**，之后，重新选择进程进行调度。软中断是由指令执行过程中发出的中断，但是并没有中断向量表，而是有对应**的散转**表，查找对应的**中断号**，转中断服务程序，之后的和硬中断相同。

系统调用是软中断的一种。无论如何，发生中断时，要从目态转向管态。



22

系统调用和库函数调用的区别：

1. 系统调用，最底层，面向硬件；库函数调用面向开发，相当于预先定义好的 API；
2. 系统调用是基于操作系统的，不同的操作系统之间是不同的，不具有可移植性；而库函数调用移植性良好；
3. 系统调用需在用户空间和内核空间进行切换，开销较大；库函数属于过程调用，调用开销较小；
4. 库函数调用最终还是通过系统调用来实现的，结合缓冲区技术，使得库函数可以大大减少调用次数，节省开销；系统调用的是系统内核的服务。

3. 进程、线程的概念以及区别？多进程和多线程？进程间的通信方式？★★★★★★

一、进程与线程的区别

进程是具有一定功能的程序，是系统进行**资源分配调度的**一个独立单位。

线程是进程的一个实体，是**处理器任务调度分配**的基本单位，线程之间基本上不拥有系统资源。

一个程序至少有一个进程，一个进程至少有一个线程，资源分配给进程，同一个进程下所有线程共享该进程的资源。

根本区别：进程是操作系统资源分配的基本单位，而线程是处理器任务调度和执行的基本单位

资源开销：每个进程都有独立的代码和数据空间（程序上下文），程序之间的切换会有较大的开销；线程可以看做轻量级的进程，同一类线程共享代码和数据空间，每个线程都有自己独立的运行栈和程序计数器（PC），线程之间切换的开销小。

包含关系：如果一个进程内有多个线程，则执行过程不是一条线的，而是多条线（线程）共同完成的；线程是进程的一部分，所以线程也被称为轻权进程或者轻量级进程。

内存分配：同一进程的线程共享本进程的地址空间和资源，而进程之间的地址空间和资源是相互独立的

影响关系：一个进程崩溃后，在保护模式下不会对其他进程产生影响，但是一个线程崩溃整个进程都死掉。所以多进程要比多线程健壮。

执行过程：每个独立的进程有程序运行的入口、顺序执行序列和程序出口。但是线程不能独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制，两者均可并发执行

(18 条消息) 进程和线程的区别(超详细) ThinkWon 的博客-CSDN 博客 进程和线程的区别

- **匿名管道通信：**管道是一种半双工的通信方式，数据只能单向流动，而且只能在具有亲缘关系的进程间使用。进程的亲缘关系通常是指父子进程关系。
- **高级管道通信：**将另一个程序当做一个新的进程在当前程序进程中启动，则它算是当前程序的子进程，这种方式我们成为高级管道方式。
- **有名管道通信 ：** 有名管道也是半双工的通信方式，但是它允许无亲缘关系进程间的通信。
- **消息队列通信 ：** 消息队列是由消息的链表，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺点。
- **信号量通信 ：** 信号量是一个计数器，可以用来控制多个进程对共享资源的访问。它常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。因此，主要作为进程间以及同一进程内不同线程之间的同步手段。
- **信号：** 信号是一种比较复杂的通信方式，用于通知接收进程某个事件已经发生。
- **共享内存通信：共享内存就是映射一段能被其他进程所访问的内存，这段共**

享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的 IPC 方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号两，配合使用，来实现进程间的同步和通信。

- **套接字通信** ： 套接口也是一种**进程间通信机制**，与其他通信机制不同的是，它可用于不同机器间的进程通信。

二、线程哪些资源共享？哪些资源不共享？

共享：堆、全局变量、静态变量、文件等共用资源

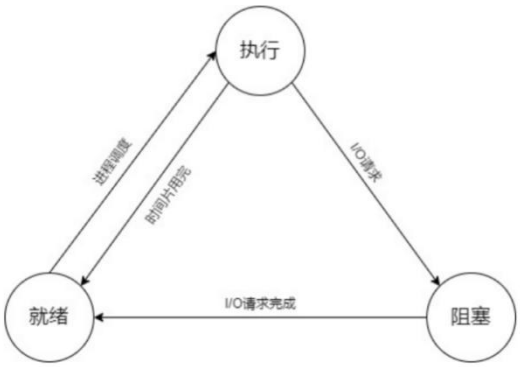
独享：栈、寄存器、pc（程序计数器）

进程间的通信方式：https://blog.Csdn.net/zhaohong_bo/article/details/89552188

4. 进程有哪几种状态，状态之间的转换、进程调度策略？★★★★

一、进程的三种状态

- **就绪（Ready）状态** 进程已分配到除 CPU 以外的所有必要资源，只要获得处理机便可立即执行。
- **执行（Running）状态** 进程已获得处理机，其程序正在处理机上执行。
- **阻塞（Blocked）状态** 正在执行的程序，由于等待某个事件发生而无法执行时，便放弃处理机而处于阻塞状态。引起进程阻塞的原因可能是**等待 I/O 完成、申请缓冲区不能满足、等待信号**等。



Q1：为什么在转换图中没有就绪到阻塞和阻塞到执行的转换方向？

就绪状态进程没有占有处理机，即不经过执行，其状态就不会改变；阻塞状态进程唤醒后要先进入到就绪队列，才会被调度程序选中，进行执行状态

二、进程调度策略

1.**先来先服务调度算法：**先来先服务(FCFS)调度算法是一种最简单的调度算法，该算法既可用于**作业调度，也可用于进程调度**。当在作业调度中采用该算法时，每次调度都是从**后备作业队列**中选择一个或多个最先进入该队列的作业，将它们调入内存，为它们分配资源、创建进程，然后放入就绪队列。在进程调度中采用 FCFS 算法时，则每次调度是从就绪队列中选择一个最先进入该队列的进程，为之分配处理机，使之投入运行。该进程**一直运行到完成或发生某事件而阻塞**后才放弃处理机。

2.**短作业(进程)优先调度算法：**短作业(进程)优先调度算法 SJ(P)F，是指对短作业或短进程优先调度的算法。它们可以分别用于**作业调度和进程调度**。短作业优先(SJF)的调度算法是从后备队列中选择一个或若干个估计运行时间最短的作业，将它们调入内存运行。而短进程优先(SPF)调度算法则是从就绪队列中选出一个估计运行时间最短的进程，将处理机分配给它，使它立即执行并一直执行到完成，或发生某事件而被阻塞放弃处理机时再重新调度。

3.**高优先权优先调度算法：**为了**照顾紧迫型作业**，使之在进入系统后便获得优先处理，引入了最高优先权优先(FPF)调度算法。此算法常被用于**批处理系统**中，作为**作业调度算法**，也作为多种操作系统中的**进程调度算法**，还可用于**实时系统**中。当把该算法用于作业调度时，系统将从后备队列中选择若干个优先权最高的作业装入内存。当用于进程调度时，该算法是把处理机分配给就绪队列中优先权最高的进程，这时，又可进一步把该算法分成如下两种。

3.1) **非抢占式优先权算法：**在这种方式下，系统一旦把处理机分配给就绪队列中优先权最高的进程后，**该进程便一直执行下去**，直至完成；或因发生某事件使该进程放弃处理机时，系统方可再将处理机重新分配给另一优先权最高的进程。这种调度算法主要用于批处理系统中；也可用于某些对**实时性要求不严**的实时系统中。

3.2) **抢占式优先权调度算法：**在这种方式下，系统同样是把处理机分配给优先权最高的进程，使之执行。但在其执行期间，**只要又出现了另一个其优先权更高的进程，进程调度程序就立即停止**当前进程(原优先权最高的进程)的执行，重新将处理机分配给新到的优先权最高的进程。因此，在采用这种调度算法时，是每当系统中出现一个新的就绪进程 i 时，就将其优先权 Pi 与正在执行的进程

j 的优先权 Pj 进行比较。如果 Pi≤Pj，原进程 Pj 便继续执行；但如果是 Pi>Pj，则立即停止 Pj 的执行，做进程切换，使 i 进程投入执行。显然，这种抢占式的优先权调度算法能更好地满足紧迫作业的要求，故而常用于要求**比较严格的实时系统**中，以及对**性能要求较高的批处理和分时系统**中。

4.**高响应比优先调度算法**：在批处理系统中，短作业优先算法是一种比较好的算法，其主要的不足之处是长作业的运行得不到保证。如果我们能为每个作业引入前面所述的动态优先权，并使作业的优先级随着等待时间的增加而以速率 a 提高，则长作业在等待一定的时间后，必然有机会分配到处理机。该优先权的变化规律可描述为：

$$R_p = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$$

在利用该算法时，每要进行调度之前，都须先做**响应比的计算**，这会增加系统开销。

5.**时间片轮转法**：在早期的时间片轮转法中，系统将所有的就绪进程按先来先服务的原则排成一个队列，每次调度时，把 CPU 分配给队首进程，并令其执行一个时间片。时间片的大小从几 ms 到几百 ms。当执行的时间片用完时，由一个计时器发出时钟中断请求，调度程序便据此信号来停止该进程的执行，并将它送往就绪队列的末尾；然后，再把处理机分配给就绪队列中新的队首进程，同时也让它执行一个时间片。这样就可以保证就绪队列中的所有进程在一给定的时间内均能获得一时间片的处理机执行时间。换言之，系统能在给定的时间内响应所有用户的请求。

6.**多级反馈队列调度算法**：前面介绍的各种用作进程调度的算法都有一定的局限性。如短进程优先的调度算法，仅照顾了短进程而忽略了长进程，而且如果并未指明进程的长度，则短进程优先和基于进程长度的抢占式调度算法都将无法使用。而多级反馈队列调度算法则不必事先知道各种进程所需的执行时间，而且还可以满足各种类型进程的需要，因而它是目前被公认的一种较好的进程调度算法。在采用多级反馈队列调度算法的系统中，调度算法的实施过程如下所述。

(1) **应设置多个就绪队列，并为各个队列赋予不同的优先级**。第一个队列的优先级最高，第二个队列次之，其余各队列的优先权逐个降低。该算法赋予各个

队列中进程执行时间片的大小也各不相同，**在优先权愈高的队列中，为每个进程所规定的执行时间片就愈小**。例如，第二个队列的时间片要比第一个队列的时间片长一倍，……，第 i+1 个队列的时间片要比第 i 个队列的时间片长一倍。

(2)当一个新进程进入内存后，首先将**它放入第一队列的末尾，按 FCFS 原则排队等待调度**。当轮到该进程执行时，如它能在该时间片内完成，便可准备撤离系统；如果它在一个时间片结束时尚未完成，调度程序便将该进程转入第二队列的末尾，再同样地按 FCFS 原则等待调度执行；如果它在第二队列中运行一个时间片后仍未完成，再依次将它放入第三队列，……，如此下去，当一个长作业(进程)从第一队列依次降到第 n 队列后，在**第 n 队列便采取按时间片轮转**的方式运行。

(3) 仅当第一队列空闲时，调度程序才调度第二队列中的进程运行；仅当第 1～(i-1)队列均空时，才会调度第 i 队列中的进程运行。如果处理机正在第 i 队列中为某进程服务时，又有新进程进入优先权较高的队列(第 1～(i-1)中的任何一个队列)，则此时**新进程将抢占正在运行进程的处理机**，即由调度程序把正在运行的进程放回到第 i 队列的末尾，把处理机分配给新到的高优先权进程。

三、*读写者问题是用进程实现的还是线程实现的？文件系统中文件是如何组织的？

读写者问题实现：

读写者问题是经典进程同步问题，是用进程实现的。 利用信号量机制来实现互斥访问（整型信号量、记录型信号量、AND 型信号量、信号量集）

5. 什么是死锁？死锁产生的四个必要条件？如何预防死锁？★★★★★★

一、什么是死锁

死锁是指多个进程因竞争资源而造成的一种僵局（互相等待），若无外力作用，这些进程都将无法向前推进。

二、产生死锁的原因：

- **竞争不可抢占型资源**（例如：系统中只有一台打印机，可供进程 P1 使用，

假定 P1 已占用了打印机，若 P2 继续要求打印机打印将阻塞）

- **竞争可消耗型资源**（可消耗型资源包括硬件中断、信号、消息、缓冲区内的消息等），通常消息通信顺序进行不当，则会产生死锁
- **进程推进顺序不当**（若 P1 保持了资源 R1，P2 保持了资源 R2，系统处于不安全状态，因为这两个进程再向前推进，便可能发生死锁。

当 P1 运行到 P1，Request 时，将因 R2 已被 P2 占用而阻塞；当 P2 运行到 P2：Request 时，也将因 R1 已被 P1 占用而阻塞，于是发生进程死锁）

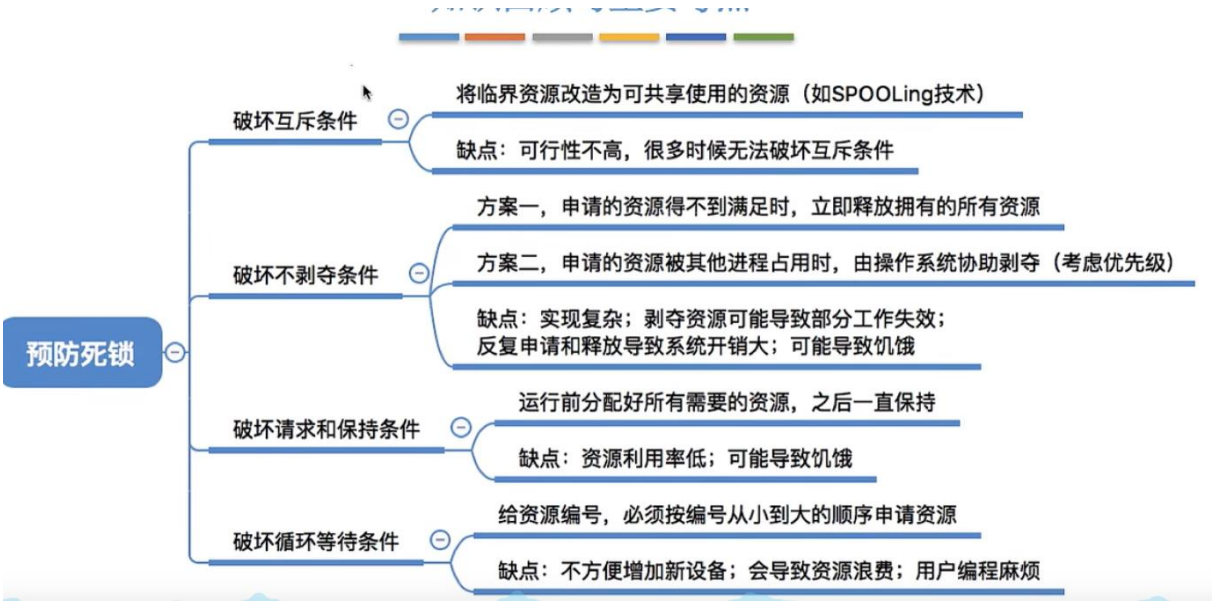
三、死锁产生的四个必要条件

- **互斥条件：**进程要求对所分配的资源进行排它性控制，即在一段时间内某资源仅为一进程所占用。
- **请求和保持条件：**当进程因请求资源而阻塞时，对已获得的资源保持不放。
- **不剥夺条件：**进程已获得的资源在未使用完之前，不能剥夺，只能在使用完时由自己释放。
- **环路等待条件：在发生死锁时，必然存在一个进程--资源的环形链。**

四、如何预防死锁

- 资源一次性分配：一次性分配所有资源，这样就不会再有请求了：（破坏请求条件）
- 允许程序获得运行初期条件的所有资源后开始运行，过程中逐步释放给自己的已用毕的资源，然后再请求新的资源（破坏请保持条件）
- 可剥夺资源：即当某进程获得了部分资源，但得不到其它资源，则释放已占有的资源（破坏不可剥夺条件）
- 资源有序分配法：系统给每类资源赋予一个编号，每一个进程**按编号递增的顺序请求资源(当你持有高等级的资源又申请低等级的资源时，必须要先释放掉占有的高级资源，从而破坏了循环等待的条件)**，释放则相反（破坏环路等待条件）
- 原文链接：

  https://blog.csdn.net/hd12370/article/details/82814348



四、如何预防死锁

- 资源一次性分配：一次性分配所有资源，这样就不会再有请求了：（破坏请求条件）
- **允许程序获得运行初期条件的所有资源后开始运行，**过程中逐步释放给自己的已用毕的资源，然后再请求新的资源（破坏保持条件）**获得运行需要的所有资源**
- **可剥夺资源：**即当某进程获得了部分资源，但得不到其他资源，则释放已占有的资源（破坏不可剥夺条件）
- **资源有序分配法：**系统给每类资源赋予一个编号，每一个进程按编号递增的顺序请求资源，释放则相反（破坏环路等待条件）

原文链接：https://blog.csdn.net/hd12370/article/details/82814348

6. 哲学家进餐有哪些实现方式？★★★★

哲学家进餐问题的核心是保证至少有一位哲学家能拿到两只筷子就餐后释放筷子。

（1）最多只允许 n-1 个哲学家拿起筷子就餐。

（2）资源分级算法，奇数号哲学家先拿左手边的筷子，偶数号的哲学家先拿右手边的筷子。

（3）设立规则，当一位哲学家拿起一支筷子时，另一个筷子无法得到，则放下刚刚拿起的筷子。

（4）服务生算法，一次只允许一名哲学家进餐，等到这名哲学家进餐完毕后才允许其他哲学家进餐。

7. 简述下银行家算法★★★★

银行家算法是一种最有代表性地避免死锁的算法。在避免死锁方法中允许进程动态地申请资源，但系统在进行资源分配之前，应先计算此次分配资源的安全性，若分配不会导致系统进入不安全状态，则分配，否则等待。

为实现银行家算法，系统必须设置若干数据结构。

银行家算法中的数据结构

为了实现银行家算法，必须设置以下四个数据结构：

（1）**可利用资源向量** Available：其初始值是系统中所配置的该类全部可用资源的数目。

（2）**最大需求矩阵 Max**：它定义了系统中 n 个进程中的每一个进程对 m 类资源的最大需求。

（3）**分配矩阵 Allocation**：它定义了系统中每一类资源当前已分配给每一个进程的资源数。

（4）**需求矩阵 Need**：用一表示每一个进程尚需的各类资源数。

银行家算法检查的步骤

Request(i)是进程 P(i)的请求向量。如果 Requst(i)[j]=K，表示进程 P(i)需要 K 个 R(j)类型的资源。

（1）如果 Request(i)[j] <= Need[i，j]，便转向步骤 2，否则出错；

（2）如果 Request(i)[j] <= Available[j]，便转向步骤 3，否则出错；

（3）系统试探着把资源分配给进程 P，并修改下面数据结构中的数值：

（非常重要）

Available[j] = Avaliable[j] - Request(i)[j];

Allocation[i，j] = Allocation[i，j] + Request(i)[j];

Need[i，j] = Need[i，j] - Request(i)[j];

（4）执行**安全性算法**，检查此次资源分配后系统是否处于安全状态*。

安全则分配，否则分配作废。

原文链接：https://blog.csdn.net/qq_34649947/article/details/70224868

8. 介绍一下几种常见的进程调度算法及其流程（FCFS，SJF，剩余短作业优先，优先级调度，轮转法，多级反馈队列等等）★★★★★★

参考链接：(21 条消息) 常用的进程调度算法_aiguangyuan 的博客-CSDN 博客_进程调度算法

9. 分页的作用，好处？和分段有什么区别？★★★

- **页是信息的物理单位，分页是为了实现非连续分配，以便解决内存碎片问题，**或者说分页是由于系统管理的需要。**段是信息的逻辑单位**，它含有一组意义相对完整的信息，分段的目的是更好地**实现共享**，满足用户的需要。
- 页的大小固定且由系统确定，**将逻辑地址划分为页号和页内地址**是由机器硬件实现的，而**段的长度却不固定**，决定于用户所编写的程序，通常由编译程序在对源程序进行编译时根据信息的性质来划分。
- 分页的作业地址空间是一维的；分段的地址空间是二维的。

10. 内存分配有哪些机制？（JVM 的内存管理及垃圾回收算法）★★★

单一连续分配、固定分区分配、动态分区分配、基于顺序搜索的动态分区分配 1. 首次适应算法、2. 循环首次适应算法、3. 最佳适应算法、4. 最坏适应算法。

- 基于索引搜索的动态分区分配算法：1. 快速适应算法、2. 伙伴系统、3. 哈希算法
- 动态可重定位分区分配

11. 什么是虚拟内存？什么是共享内存？★★★

问题：

- 有的作业很大，所要求的内存空间超过了内存总容量，作业不能全部装入内

存，致使该作业无法运行。

- 大量作业要求运行，但是内存容量不足以容纳所有的这些作业，只能装入少量作业

解决：

- **虚拟内存技术：**从逻辑上扩充内存容量，在运行之前并不是全部装入内存，而是将需要运行的页或段装入内存，如果要访问的页或段未装入内存，在发出缺页（段）中断请求。
- 共享内存是最快速的进程间通信机制。操作系统在几个进程的地址空间上映射一段内存，然后这几个进程可以在不需要调用操作系统函数的情况下在那段内存上进行读/写操作
- 虚拟内存是计算机系统内存管理的一种技术。它使得应用程序认为它拥有连续可用的内存（一个连续完整的地址空间），而实际上，它通常是被分隔成多个物理内存碎片，还有部分暂时存储在外部磁盘存储器上，在需要时进行数据交换。与没有使用虚拟内存技术的系统相比，使用这种技术的系统使得大型程序的编写变得更容易，对真正的物理内存（例如 RAM）的使用也更有效率。

原文链接：https://blog.csdn.net/zhanyd/article/details/102987669
http://blog.sina.com.cn/s/blog_a9cdad020102wxzw.html

12. 有哪些页面置换算法？★★★★

如何选择页面进入内存，调出内存的方式

- **最佳置换算法（OPT）：**每次选择淘汰的页面将是以后永不使用，或者在最长时间内不再被访问的页面，这样可以保证最低的缺页率。
- **先进先出置换算法（FIFO）：**每次选择淘汰的页面是最早进入内存的页面。
- **最近最久未使用置换算法：**每次淘汰的页面是最近最久未使用的页面
- **时钟置换算法：**为每个页面设置一个访问位，再将**内存中的页面**都通过链接指针链接成一个循环队列。当某个页被访问时，其访问位置 1。当需要淘汰一个页面时，只需检查页的访问位。**如果是 0，就选择该页换出；如果是 1，暂不换出，**将访问位改为 0，继续检查下一个页面，若第一轮扫描中所有的页面都是 1，则将这些页面的访问位一次置为 0 后，再进行第二轮扫描（第

二轮扫描中一定会有访问位为 0 的页面，因此简单的 CLOCK 算法选择一个淘汰页面最多会经过两轮扫描）。

- **改进型时钟置换算法：有两个位，访问位和修改位，用于判断是否访问和修改**

第一轮：从当前位置开始扫描第一个的页用于替换，本轮扫描不修改任何标志位。

第二轮：若第一轮扫描失败，则重新扫描，查找第一个的页用于替换。本轮将所有扫描过的页访问位设为 0。

第三轮：若第二轮扫描失败，则重新扫描，查找第一个的页用于替换。本轮扫描不修改任何标志位。

第四轮：若第三轮扫描失败，则重新扫描，查找第一个的页用于替换。

> 第一轮：从当前位置开始扫描第一个（0,0）的页用于替换，本轮扫描不修改任何标志位。
> 第二轮：若第一轮扫描失败，则重新扫描，查找第一个（0,1）的页用于替换。本轮将所有扫描的过的页访问位设为0。
> 第三轮：若第二轮扫描失败，则重新扫描，查找第一个（0,0）的页用于替换。本轮扫描不修改任何标志位。
> 第四轮：若第三轮扫描失败，则重新扫描，查找第一个（0,1）的页用于替换。

原文链接：https://www.jianshu.com/p/18285ecffbfb

13. 说一说操作系统中缓冲区溢出怎么处理★★★

- **缓冲区溢出**

当计算机程序向缓冲区内填充的数据位数超过了缓冲区本身的容量。溢出的数据覆盖在合法数据上

理想情况是，程序检查数据长度并且不允许输入超过缓冲区长度的字符串。但是绝大多数程序都会假设数据长度总是与所分配的存储空间相匹配，这就为缓冲区溢出埋下隐患。操作系统所使用的缓冲区又被称为堆栈，在各个操作进程之间，指令被临时存储在堆栈当中，堆栈也会出现缓冲区溢出。

- **上溢**

当一个超长的数据进入到缓冲区时，超出部分被写入上级缓冲区，

上级缓冲区存放的可能是数据、上一条指令的指针，或者是其他程序的输出内容，这些内容都被覆盖或者破坏掉。可见一小部分数据或者一套指令的溢出就可能导致一个程序或者操作系统崩溃。

- **下溢**

当一个超长的数据进入到缓冲区时，超出部分被写入下级缓冲区，

下级缓冲区存放的是下一条指令的指针，或者是其他程序的输出内容。

缓冲区攻击的日渐泛滥，微软并未任其张扬，陆陆续续推出了各种保存措施。其中重要的有 GS，SafeSeh，ASLR，DEP 等，接下来我们针对这些措施进行原理分析：

- **基于探测方法（canary）的防御**

非执行的堆栈防御

①.GS 保护原理：

通过 VC++编译器在函数前后添加额外的处理代码，前部分用于由伪随机数生成的 cookie 并放入。data 节段，当本地变量初始化，就会向栈中插入 cookie，它位于局部变量和返回地址之间在缓冲区溢出利用时，如果将恶意代码从局部变量覆盖到返回地址，那么自然就会覆写 cookie，当检测到与原始 cookie 不同时，就会触发异常，最后终止进程。

②.SafeSeh 保护

为了防止 SEH 节点被攻击者恶意利用，微软在. net 编译器中加入/sdeseh 编译选项引入 SafeSEH 技术。编译器在编译时将 PE 文件所有合法的异常处理例程的地址解析出来制成一张表，放在 PE 文件的数据块(LQAJ)一 CON—FIG)中，并使用 shareuser 内存中的一个随机数加密，用于匹配检查。

如果该 PE 文件不支持 safesEH，则表的地址为 0。当 PE 文件被系统加载后，表中的内容被加密保存到 ntdll. dll 模块的某个数据区。在 PE 文件运行期间，如果发生异常需要调用异常处理例程，系统会逐个检查该例程在表中是否有记录：如果没有则说明该例程非法，进而不执行该异常例程。

③.ASLR 保护

ASLR（地址空间布局随机化）技术的主要功能是通过对系统关键地址的随机化，防止攻击者在堆栈溢出后利用固定的地址定位到恶意代码并加以运行。

④.DEP 保护

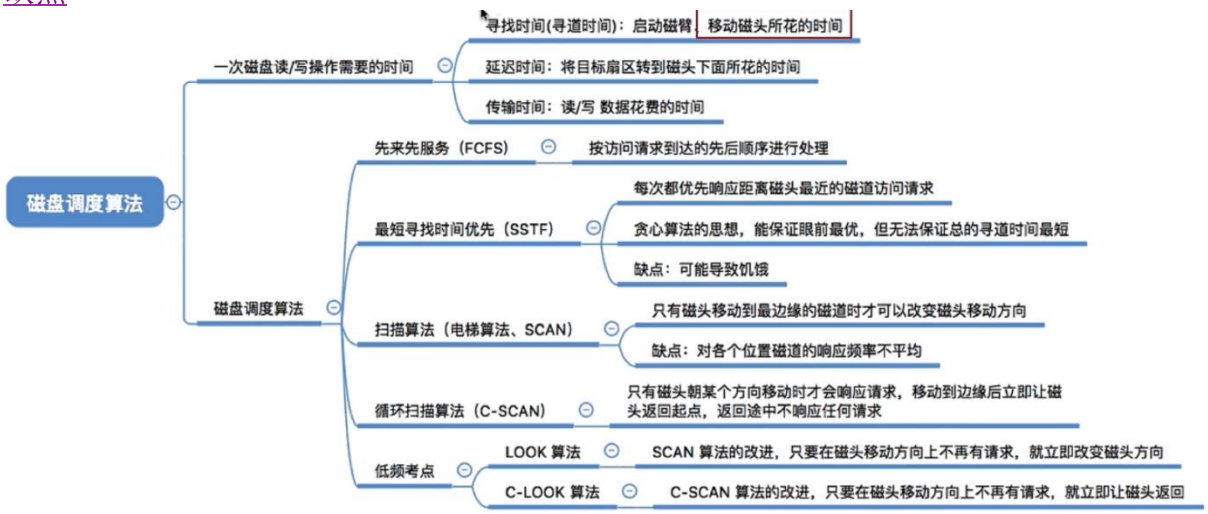
数据执行保护（DEP）是一套软硬件技术，能够在内存上执行额外检查以防止在不可运行的内存区域上执行代码

这些保护机制的出现，一度使得攻击难度大大增加，但攻击者们也不是吃素的，他们也在研究绕过这些保护的办法。所谓千里之堤溃于蚁穴，对于上面的保护，攻击者只要找到了它们的一个弱点，便可以突破所有防线，实现攻击。攻击与防护，不断在对抗，可以预见，在未来一段时间，这种游戏还将持续上演。

但是，现在的漏洞门槛比十年前高了 N 倍，笔者在实际中也常常遇到有漏洞却不能利用的情况。但是也不要灰心，老的技术被淘汰，新的技术又会出来，本属正常，但像缓冲区溢出这种经典的东西，无论什么时候，都值得我们学习。

14. 磁盘调度算法以及磁盘空间存储管理？★★★★

(38 条消息) 浅谈 磁盘调度算法 辉仔的博客-CSDN 博客 磁盘调度算法优缺点



1. **FCFS**：先来先服务 FCFS（First come, First Serve）：是一种自然公平的调度策略。先来先到，无特权用户。FCFS 算法根据进程请求访问磁盘的先后顺序进行调度，这是一种最简单的调度算法。优点：公平，若访问进程数量少，且请求都是访问块聚的文件扇区，性能较好，否则接近于随机调度。

2. **STF**：短任务优先 STF（Shortest Task First）:就是谁的磁盘读写数据量少，谁就优先。由于磁盘的访问时间**主要取决于寻道和旋转延迟**，因此读写的数据量对整个磁盘读写的时间影响并不大。因此此种策略的意义不大。

3. **SSF：短寻道优先** SSF：（Shortest Seek First）则是考虑当前磁头离

谁的数据最近，谁就优先。由于寻道在磁盘访问时间中占得比重最大，此种策略似乎正中要害，能够缩短磁盘访问时间。

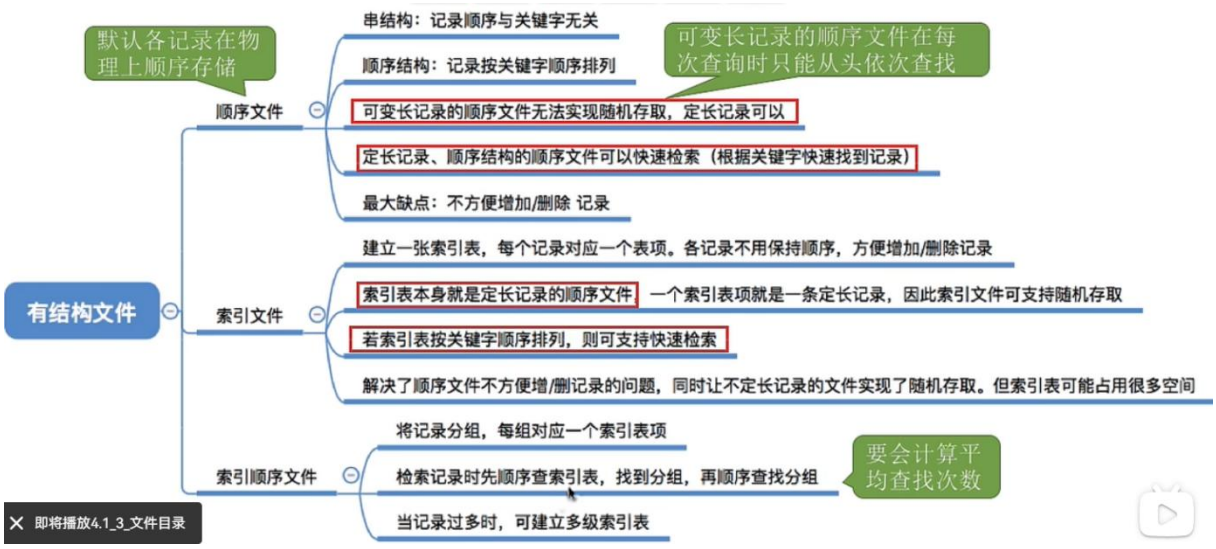
4. **ES：电梯调度策略** ES：（Elevator Scheduling），又称扫描算法（SCAN），**先满足一个方向上所有请求，再满足相反方向的所有请求**，循环往复。磁头向每个方向运动时（由里向外和由外向里），皆扫描到头。

5. **ESLA：提前查看电梯调度** ESLA（Elevator Scheduling with Look Ahead），**如果一个方向的请求全部满足后，立即进行反转，无需扫描到底**。这种算法就是每次往某个方向移动时，必须确保该方向还有请求未被满足。否则即刻调转方向，这样效率将得到提高。

6. **OWES：单向电梯调度** OWES（One Way Elevator Scheduling）。对于提前查看电梯法的改进，就是单向电梯调度，**即只向一个方向扫描**。当该方向没有剩余请求时，则回到 0 道，再进行同样的扫描。如果在磁盘请求与图 5 同样的情况下，使用单向电梯调度的总寻道数为：1 + 4 + 16 + 8 = 29，比提前查看电梯法节省 23 个磁道的寻道时间。

15. **文件系统中文件是如何组织的？★★**

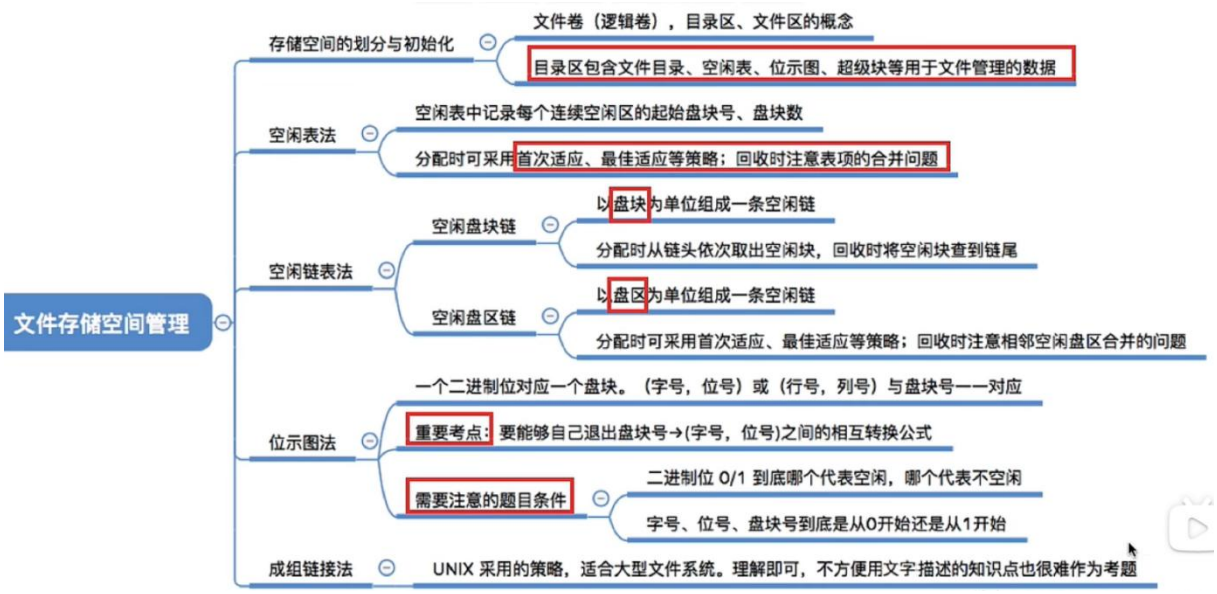
逻辑结构：研究的是文件内部是如何组织的



目录分配：

物理结构：研究的是文件在物理磁盘上是如何组织的

		How?	目录项内容	优点	缺点
顺序分配		为文件分配的必须是连续的磁盘块	起始块号、文件长度	顺序存取速度快，支持随机访问	会产生碎片，不利于文件拓展
链接分配	隐式链接	除文件的最后一个盘块之外，每个盘块中都存有指向下一个盘块的指针	起始块号、结束块号	可解决碎片问题，外存利用率高，文件拓展实现方便	只能顺序访问，不能随机访问。
	显式链接	建立一张文件分配表(FAT)，显式记录盘块的先后关系（开机后FAT常驻内存）	起始块号	除了拥有隐式链接的优点之外，还可通过查询内存中的FAT实现随机访问	FAT需要占用一定的存储空间
索引分配		为文件数据块建立索引表。若文件太大，可采用链接方案、多层索引、混合索引	链接方案记录的是第一个索引块的块号，多层/混合索引记录的是顶级索引块的块号	支持随机访问，易于实现文件的拓展	索引表需占用一定的存储空间。访问数据块前需要先读入索引块。若采用链接方案，查找索引块时可能需要很多次读磁盘操作。



- **文件的逻辑结构：**从用户出发所观察到的文件组织形式，即文件是由一系列

的逻辑记录组成的。

- **文件的物理结构：**（又称文件的存储结构），指系统文件存储在外存上的形式
- **文件的逻辑组织**通常分为两种形式，**即有结构文件（记录文件）和无结构文件（字符流文件）**。

有结构文件又称作记录式文件，可分为：**定长记录文件和变长记录文件**两种，**定长记录处理方便，开销小，被广泛用于数据处理中。**无结构文件是指**文件内部不再划分记录**

◆ 常用的记录式结构有：连续结构、多重结构、转置结构和顺序结构。

◆ 常用的存取方法有：顺序存取法、随机存取法（直接存取法）和按键存取法。

◆ **常用的文件物理结构有：连续文件、串联文件和索引文件**

✓ **文件的逻辑组织文件的逻辑组织通常分为两种形式，即有结构文件（记录文件）和无结构文件（字符流文件）。**

1）有结构文件又称作记录式文件，它在逻辑上可被看成一组连续记录的集合，即文件是由若干个相关的记录组成。每个记录是一组相关的数据集合，用于描述一个对象某个方面的属性。记录式文件按其记录的长度是否相同又可分为：定长记录文件和变长记录文件两种。

定长记录文件：指文件中所有记录的长度都相同。文件的长度可用记录的数目来表示。定长记录处理方便，开销小，被广泛用于数据处理中。

变长记录文件：指文件中各记录的长度不相同。在处理之前每个记录的长度是已知的。

2）无结构文件**是指文件内部不再划分记录**，它是由一组相关信息组成的有序字符流，即流式文件，其长度直接按字节计算。如大量的源程序、可执行程序、库函数等采用的文件形式是无结构文件形式。在 UNIX 系统中，所有的普通文件都被看作是流式文件，系统不对文件进行格式处理。

常用的记录式结构有：连续结构、多重结构、转置结构和顺序结构。

常用的存取方法有顺序存取法、随机存取法（直接存取法）和按键存取法。文件的。**我自己看了王道觉得是顺序、链接、索引三种方式。**

✓ **常用的文件物理结构有连续文件、串联文件和索引文件。**

1）连续文件（又称做顺序文件）是基于磁带设备的最简单的物理文件结构，它是把一个逻辑上连续的文件信息存放在连续编号的物理块（或物理记录）中。连续文件的优点是在顺序存取时速度较快，常用于存放系统文件，如操作系统文件、编译程序文件和其他由系统提供的实用程序文件，因为这类文件往往被从头至尾依次存取。但连续文件也存在如下缺点：

（1）要求建立文件时就确定它的长度，以此来分配相应的存储空间，这往往很难实现。

（2）不便于文件的动态扩充。

（3）可能出现外部碎片，就是在存储介质上存在很多空闲块，但它们都不连续，无法被连续的文件使用，从而造成浪费。

2）串联文件为克服连续文件的缺点，可把一个逻辑上连续的文件分散存放在不同的物理块中，这些物理块不要求连续，也不必规则排列。为了使系统能找到下一个逻辑块所在的物理块，可在各物理块中设立一个指针（称为连接字），它指示该文件的下一个物理块。串连文件克服了连续文件的缺点，但它又带来新的问题：

（1）一般仅适于对信息的顺序访问，而不利于对文件的随机存取。

（2）每个物理块上增加一个连接字，为信息管理添加了一些麻烦。FAT 格式通过把文件分配表（FAT，File Allocation Table ）放在一个内存表格中的方式加以克服串联文件的缺点，**一个磁盘仅设置一张 FAT，开机时，将 FAT 读入内存，并常驻内存。**

3）索引文件 索引文件是实现非连续分配的另一种方案：系统为每个文件建立一个索引表。其中的表项指出存放该文件的各个物理块号，而整个索引表由文件说明项指出。这种结构除了具备串连文件的优点之外，还克服了它的缺点。它可以方便地进行随机存取。但是这种组织形式需要增加索引表带来的空间开销。如果这些表格仅放在盘上，那么在存取文件时首先得取出索引表，然后才能查表、得到物理块号。这样就至少增加了一次访盘操作，从而降低了存取文件的速度，加重了 I/O 负担。一种改进办法是同时把索引表部分或全部地放入内存。这是以内存空间为代价来换取存取速度的改善。 树型目录结构树型目录结构可能是目录结构中，考的比较多的，考得也简单。

原文链接 https://zhidao.Baidu.com/question/1946134093877936908.html

16. **动态分区分配算法？★★**

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找适合的分区	空闲分区以地址递增次序排列	综合看性能最好。***算法开销小***，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片；***算法开销大***，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小碎片	大分区容易被用完，不利于大进程；***算法开销大***（原因同上）
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列（可排列成循环链表）	不用每次都从低地址的小分区开始检索。***算法开销小***（原因同首次适应算法）	会使高地址的大分区也被用完

17. **IO 控制方式？★★**

	完成一次读/写的过程	CPU干预频率	每次I/O的数据传输单位	数据流向	优缺点
程序直接控制方式	CPU发出I/O命令后需要不断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的优点都是解决了上一阶段的最大缺点。 总体来说，整个发展过程就是要尽量减少CPU对I/O过程的干预，把CPU从繁杂的I/O控制事务中解脱出来，以便更多地去完成数据处理任务。
中断驱动方式	CPU发出I/O命令后可以做其他事，本次I/O完成后设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以做其他事，本次I/O完成后DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方式	CPU发出I/O命令后可以做其他事。通道会执行通道程序以完成I/O，完成后通道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

2.4. 计算机组成原理

1. 冯诺依曼机的体系结构★★★★★

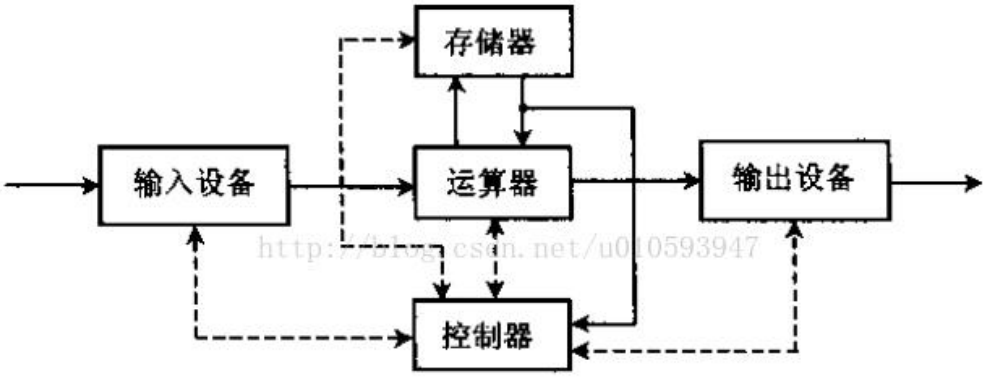


图 1.7 典型的冯·诺依曼计算机结构框图

主要由五大部件组成

1. 存储器用来存放数据和程序

2. 运算器主要运行算术运算和逻辑运算，并将中间结果暂存到运算器中

3.控制器主要用来控制和指挥程序和数据的输入运行，以及处理运算结果

4.输入设备用来将人们熟悉的信息形式转换为机器能够识别的信息形式，常见的有键盘，鼠标等

5.输出设备可以将机器运算结果转换为人们熟悉的信息形式，如打印机输出，显示器输出等

2. 衡量计算机性能指标★★★★

1、**吞吐量**：表征一台计算机在某一时间间隔内能够处理的信息量，单位是字节/秒。

2、**响应时间**：表征**从输入有效到系统产生响应之间的时间度量**，用时间单位来度量，例如微秒（10-6S）、纳秒（10-9S）。

3、**利用率**：表示在给定的时间间隔内，系统被实际使用的时间所占的比率，一般用百分比表示。

4、**处理机字长**：指**处理机运算器中一次能够完成二进制数运算的位数**。当前处理机的字长有 8 位、16 位、32 位、64 位。字长越长，表示计算的精度越高。

5、**总线宽度**：一般指 CPU 中运算器与存储器之间进行互连的内部总线二进制位数。

6、**存储器容量**：**存储器中所有存储单元的总数目**，通常用 KB、MB、GB、TB 来表示.其中 K=210，M=220，G=230，T=240， B=8 位（1 个字节）。

7、**存储器带宽**：**存储器的速度指标**，单位时间内从存储器读出的二进制数信息量，一般用字节数/秒表示。

8、**主频/时钟周期**：CPU 的工作节拍受主时钟控制，主时钟不断产生固定频率的时钟，主时钟的频率（f）叫 CPU 的主频。度量单位是 MHz、GHz。 主频的倒数称为 CPU 时钟周期（T），即 T=1/f，度量单位是微秒、纳秒。

9、**CPU 执行时间**：表示 CPU 执行一段程序所占用的 CPU 时间，可用下式计算：CPU 执行时间 ＝ CPU 时钟周期数 × CPU 时钟周期长 CPI：表示每条指令周期数，即执行一条指令所需的平均时钟周期数。

MIPS：每秒百万条数据。MIPS 是单位时间内的执行指令数，所以 MIPS 值越

高说明机器速度越快。 MFLOPS 是基于操作而非指令的，只能用来衡量机器浮点操作的性能，而不能体现机器的整体性能。

3. 原码、反码、补码★★★

在学习原码， 反码和补码之前， 需要先了解机器数和真值的概念。

1、机器数

一个数在计算机中的二进制表示形式， 叫做这个数的机器数。机器数是带符号的，在计算机用一个数的最高位存放符号， 正数为 0， 负数为 1。

比如，十进制中的数 +3 ，计算机字长为 8 位，转换成二进制就是 00000011。如果是 -3 ，就是 10000011 。

那么，这里的 00000011 和 10000011 就是机器数。

2、真值

因为第一位是符号位，所以机器数的形式值就不等于真正的数值。例如上面的有符号数 10000011，其最高位 1 代表负，其真正数值是 -3 而不是形式值 131（10000011 转换成十进制等于 131）。

所以，为区别起见，将带符号位的机器数对应的真正数值称为机器数的真值。

例：0000 0001 的真值 = +000 0001 = +1，1000 0001 的真值 = －000 0001 = －1

3、 原码

原码就是符号位加上真值的绝对值，即用第一位表示符号，其余位表示值。比如如果是 8 位二进制：

[+1]原 = 0000 0001 [-1]原 = 1000 0001

第一位是符号位。 因为第一位是符号位， 所以 8 位二进制数的取值范围就是：

[1111 1111 ， 0111 1111]==>[-127 ， 127]

4、 反码

反码的表示方法是：

- 正数的反码是其本身
- 负数的反码是在其原码的基础上， 符号位不变，其余各个位取反。

[+1] = [00000001]原 = [00000001]反 [-1] = [10000001]原 = [11111110]反

5、 补码

补码的表示方法是：

- 正数的补码就是其本身
- 负数的补码是在其原码的基础上，符号位不变，其余各位取反，最后+1。（即在反码的基础上+1）

[+1] = [00000001]原 = [00000001]反 = [00000001]补 [-1] = [10000001]原 = [11111110]反 = [11111111]补

4. 奇偶校验、汉明码校验，循环冗余校验★★

奇偶校验：

奇偶校验有两种校验规则：

- 奇校验：使完整编码（有效位和校验位）中的 1 的个数为奇数个；
- 偶校验：使完整编码（有效位和校验位）中的 1 的个数为偶数个

直接举例：

待编有效信息	奇校验码	偶校验码
10111010	101110100	101110101
11010010	110100101	110100100

因此，如果是奇校验，当待编有效信息的 1 为奇数个，在最后添 0，偶数个添 1，偶校验相反。

- 奇偶校验实际上就是对我们 DnDn-1……D0 进行异或运算（两两相同为 0，不同为 1），最后偶校验生成 0，奇校验生成 1，正确，反之错误。

  上面表格，第一个我们使用奇校验，第二个使用偶校验。

  第一个奇校验：1⊕0⊕1⊕1⊕1⊕0⊕1⊕0⊕0=1

  第二个偶校验：1⊕1⊕0⊕1⊕0⊕0⊕1⊕0⊕0=0

- 如果第一个数据传输过去，变成 111110100，很明显地 D7 变成了 1，这时候再进行奇偶校验

1⊕1⊕1⊕1⊕1⊕0⊕1⊕0⊕0=0

这样我们就能判断数据中出现了错误。

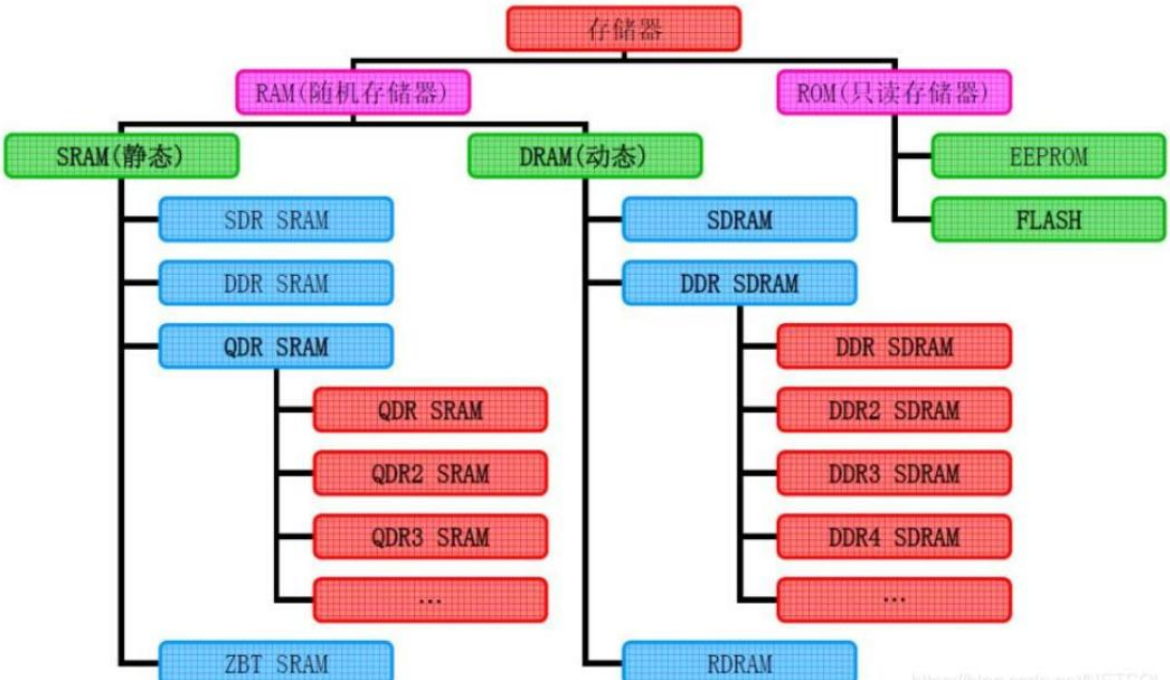
汉明码校验：

https://blog。csdn。net/zhang175gl/article/details/88637223

循环冗余校验：

https://blog。csdn。net/liyuanbhu/article/details/7882789

5. 存储器的分类（RAM、DAM 的区别）★★



常见存储器分类图示

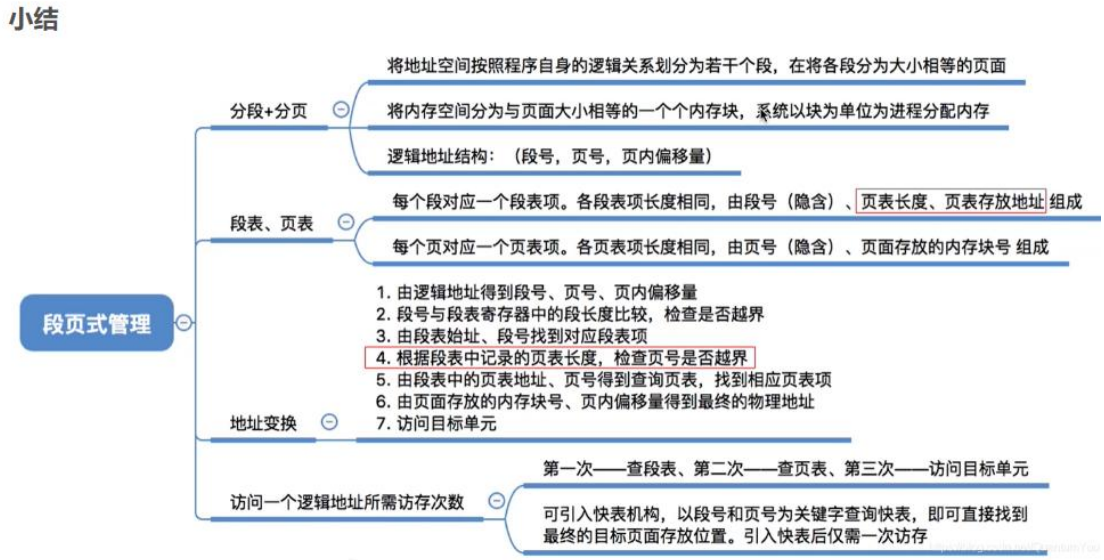
RAM：随机存取存储器是与 CPU 直接交换数据的内部存储器。它可以随时读写，而且速度很快，通常作为操作系统或其他正在运行中的程序的临时数据存储媒介。当电源关闭时 RAM 不能保留数据。如果需要保存数据，就必须把它们写入一个长期的存储设备中（例如硬盘）。RAM 和 ROM 相比，两者的最大区别是 RAM 在断电以后保存在上面的数据会自动消失，而 ROM 不会自动消失，可以长时间断电保存。

ROM：只读存储器。ROM 所存数据，一般是装入整机前事先写好的，整机工作过程中只能读出，而不像随机存储器那样能快速地、方便地加以改写。ROM 所存数据稳定，断电后所存数据也不会改变。

RAM 可以分为 SRAM（静态随机存储器）和 DRAM（动态随机存储器）。

- SRAM 它是一种具有静止存取功能的内存，不需要刷新电路即能保存它内部存储的数据。优点是速度快，不必配合内存刷新电路，可提高整体的工作效率。缺点是集成度低，功耗较大，相同的容量体积较大，而且价格较高，少量用于关键性系统以提高效率。
- DRAM 是最为常见的系统内存。DRAM 只能将数据保持很短的时间。为了保持数据，DRAM 使用电容存储，所以必须隔一段时间刷新（refresh）一次，如果存储单元没有被刷新，存储的信息就会丢失。
- SDRAM（同步动态随机存取存储器），是在 DRAM 的基础上发展而来，为 DRAM 的一种，同步是指 Memory 工作需要同步时钟，内部命令的发送与数据的传输都以时钟为基准；动态是指存储阵列需要不断的刷新来保证数据不丢失；随机是指数据不是线性依次存储，而是由指定地址进行数据读写。
- DDR SDRAM 又是在 SDRAM 的基础上发展而来，这种改进型的 DRAM 和 SDRAM 是基本一样的，不同之处在于它可以在一个时钟读写两次数据，这样就使得数据传输速度加倍了。这是目前电脑中用得最多的内存，而且它有着成本优势。

6. 段页式虚拟内存★★★



参考链接：(21 条消息) 段页式管理方式&虚拟内存_QuantumYou 的博客-CSDN 博客

7. cpu 一个指令周期的流程是什么？★★★★★

计算机执行指令的过程可以分为以下三个步骤：

1. Fetch（取指），也就是从 PC 寄存器里找到对应的指令地址，根据指令地址从内存里把具体的指令，加载到指令寄存器中，然后把 PC 寄存器自增，好在未来执行下一条指令。
2. Decode（译码），也就是根据指令寄存器里面的指令，解析成要进行什么样的操作，是 R、I、J 中的哪一种指令，具体要操作哪些寄存器、数据或者内存地址。
3. Execute（执行指令），也就是实际运行对应的 R、I、J 这些特定的指令，进行算术逻辑操作、数据传输或者直接的地址跳转。

8. 总线通讯的四种方式★★★

总线通信的四种方式_敲代码痛苦吗？的博客-CSDN 博客_总线通信控制的四种方式

同步通信，异步通信，半同步通信，分离式通信。